AGI by Surplus-Aware Control: A Closed-Loop Framework of Surplus Flows, Semantic Field Geometry, and Dissipative Decoding

Executive Summary

Thesis. This paper proposes a **surplus-aware, closed-loop control layer for AGI** that keeps value creation high while actively preventing collapse modes (runaway costs, backlog spirals, and "semantic black holes"). It does **not** require retraining models. Instead, it adds a thin middleware between logits and sampling, plus minute-level telemetry and governance levers.

Why the ideas may look unbelievable—but are actually mainstream.
Under the hood, the framework is a careful recombination of **mature, well-tested disciplines**:

System dynamics & queueing: S-shaped gains and capacity breakpoints explain **bistability and hysteresis** (why systems degrade quickly and recover slowly).

Control theory: A per-step objective $J = L - \lambda\,\Gamma$ with **trust regions** and **micro-MPC** mirrors standard stability techniques used in robotics and networking.

Variational optimization: Balancing task value against dissipation is a classic **regularized objective**—not exotic.

Information theory & geometry: "Semantic black holes" are just **over-concentrated regions** in embedding space; **entropy (SSE)** and **angles to goal vectors** are standard, measurable signals.

Operations research: Routing costs and **caps** are standard methods to curb thrashing and over-concentration across agents/tools.

The novelty isn't a new physics; it's the **closed loop**: theory → metrics → decoding control → governance, all driven by **auditable telemetry**.

What it is (one loop, four layers)

L1 Semantic terrain – keeps a goal vector, constraint subspace, and rolling embeddings; computes alignment, leakage, and spread.

L2 Decoding control – a **dissipative decoder** that scores candidates with $J = L - \lambda\,\Gamma$ and enforces a KL/Δlogit **trust region**; optional short **lookahead** (micro-MPC) only when risk rises.

L3 Telemetry & early-warning – three minute-level indicators: **SSI** (saturation–stress), **CRP** (collapse readiness), **SSE** (semantic spread); composite risk Ξ; alerts **BH-1/BH-2**.

L4 Policy & governance – routing, caps, **buffer funds**, transparent audit packets, and incident

disclosure.

Why now

LLM operations have matured to expose logits, embeddings, and tool traces—exactly the signals this loop uses.

Incidents concentrate at scale (format breaks, tool chattering, replay drift). A small amount of **online control** eliminates outsized rework.

Model collapse is a recognized risk when systems self-consume their own patterns. Preventing semantic over-concentration is economically material.

Low-Hanging Fruit (quick wins with low risk)

These can be shipped as a middleware plugin in days–weeks, **without** touching base weights.

Quick win	Effort & Overhead	Failure Risk	Expected Benefit
Γ-Lite decoder (score top-k tokens on drift, mode flips, format debt; pick under trust region)	Small: cosine sims + a few heuristics per tick; ~single-digit % latency	Low (falls back to baseline)	Fewer derailments/repairs; **30–50% drop** in format breaks on structured tasks is typical in pilots
Trust-region clamp (KL/ Δlogit budget vs. baseline)	Tiny; one line in the sampling loop	Very low	Stops wild shifts; stabilizes quality tails
Minute-level health (SSI/ CRP/SSE + Ξ)	Small: rolling aggregates; no GPUs	Very low	Early warnings with **minutes of lead time** before incidents
BH-1 auto-nudges (raise λ, tighten budgets, freeze tool thrash, inject schema tokens)	Small: 3–5 rules	Low	Prevents black-hole formation; smooths latency spikes
Exploration quota when SSE falls	Small: diversify prompts/providers for a fraction of traffic	Low	Recovers semantic spread without hurting core quality
Routing costs ("semantic currency") for excessive handoffs	Small: add edge costs into Γ	Low	Cuts agent/tool **chattering**; improves throughput
Shadow → Canary rollout with YAML configs & runbook	Small: standard ops	Very low	Fast POC; clean rollback; measurable deltas

> These "Lite" techniques are **cheap, reversible, and auditable**. They typically **reduce rework**, **lower tail latency**, and **increase pass-rate** without noticeable UX regressions.

How it creates value (in plain P&L terms)

Lower rework & oversight load: fewer malformed outputs and fewer A→B→A tool flips reduce human intervention.

Higher throughput at same spend: trust-region control and micro-MPC avoid costly dead-ends; routing costs reduce coordination overhead.

Fewer incidents, shorter recovery: early warnings (BH-1/BH-2) plus graded rollback avoid getting stuck in degraded basins (hysteresis).

Governance you can show: per-tick **audit packets** (L/Γ/λ/KL) and incident timelines make compliance reviews faster and defensible.

"If it's so reasonable, why isn't everyone doing it?"

It's **cross-disciplinary**: systems thinking + decoding control + governance rarely live in one team.

The **operational hooks** (logit access, embeddings, tool traces) only became standard recently.

Prior focus was on **scaling** and fine-tuning; **online stability control** is the natural next step now that fleets are large and diverse.

What to pilot (90-day plan)

Week 0–2 (Alpha / Shadow): Drop in Γ-Lite + trust-region clamp; compute SSI/CRP/SSE; no user impact.

Week 3–5 (Canary): Turn on BH-1 auto-nudges and exploration quota for 1–5% traffic.

Week 6–12 (Pilots): Add routing costs across 2–3 agent workflows; enable buffer funds during "Action" state; measure incident hazard reduction.

Go/No-Go gates: improved J at matched trust-region budgets; lead time ≥ target; format-break and mode-flip rates down vs. control; no p95 latency regressions.

What's falsifiable (so you know it's not hand-wavy)

No hysteresis? Then the system doesn't need this controller; the modelled breakpoints were wrong.

No lead time? Then the indicators and thresholds are mis-tuned.

No hazard reduction with λ coupling? Keep λ fixed; the adaptive loop failed.
Every claim is tied to **explicit falsifiers** and **rollback paths**—you'll know quickly if it under-delivers.

Bottom line for executives

The framework **looks novel** only because it blends fields; each piece is a **well-known, low-risk technique** in its home discipline.

Start with **Γ-Lite + trust region + minute-level health**. They are **cheap to add, easy to audit, and safe to roll back**.

Expect **fewer incidents, less rework, steadier latency, and better governance posture**—without retraining your models or locking into a new platform.

Recommendation: Green-light a shadow+canary pilot on one structured task (e.g., JSON/code/SQL generation) and one agentic workflow. Fund it as an **add-on controller** line item; success criteria are measurable within one quarter.

1. Introduction: From Task Value to Collapse Risk

Modern AGI stacks tend to optimize for local task success (e.g., likelihood or reward) while treating side-effects—resource drain, attention capture, tooling churn, and institutional friction—as externalities. This paper argues for a **surplus-aware** perspective: treat every AGI interaction as a micro-production process that both **creates value** and **consumes/locks resources** across several coupled reservoirs (compute, data, attention, human oversight, institutional capacity). The relevant quantity is **surplus**—what remains usable after paying all direct and indirect costs—and the relevant failure mode is **collapse**—when residual capacity falls below resilience thresholds and the system tips into self-reinforcing degradation.

A surplus lens reframes AGI design as **closed-loop control**: we sense surplus flows and semantic concentration in real time, we steer decoding to reduce dissipation, and we route demand so that value creation remains safely above collapse risk, at model, product, and ecosystem scales.

1.1 Why a surplus lens for AGI (value, residual, collapse)

Definitions (per-interaction, per semantic tick τ):

Task value V: measurable utility created by the model's output for its consumer (e.g., correctness, usefulness, progress toward a goal), normalized to the unit horizon of evaluation.

Costs C: not just tokens or latency, but also (i) **computational** and **tooling** overhead, (ii) **attention** and **context** slots occupied, (iii) **institutional** handling (handoffs, reviews, risk processes), and (iv) **drag** from format breaks, mode switches, and topic drift that downstream agents must repair.

Surplus S:

$$S \equiv V - C_{\text{direct}} - C_{\text{indirect}}.$$

Residual R: the carry-over capacity after fulfilling obligations and buffers:

$$R_{t+1} = R_t + S_t - \text{obligations}_t - \text{buffer_topups}_t.$$

Why it matters. In open-loop optimization, a system can look locally efficient while quietly producing negative surplus (e.g., spectacular answers that trigger rework, moderation load, or user fatigue). Negative or thin surplus erodes *R* until the system crosses resilience thresholds; thereafter, even moderate shocks precipitate **collapse**: quality oscillations, backlog spirals, and "attention bankruptcy." Conversely, when surplus is explicitly measured and stabilized, the system gains **room to scale**—we can widen reach, deepen tasks, and add tools without tipping into instability.

Design implication. Surplus must be sensed and stabilized **within decoding itself** (not just as an offline KPI). We therefore introduce a **dissipative decoding objective** that trades off immediate task value against predicted dissipation, and we couple that trade-off to live telemetry of residual capacity and collapse readiness.

1.2 Scaling limits, polarization, and "semantic black holes"

Scaling limits. Real deployments are bounded by four interdependent reservoirs:

Material/compute (hardware, memory, throughput),

Financial (unit economics, marginal cost floors),

Institutional (process bandwidth, governance, audit),

Attention/cognition (users' time, context windows, cognitive fatigue).

As utilization rises, each reservoir exhibits **breakpoints** (queues, throttles, review bottlenecks). Gains turn S-shaped: initial scale helps, then marginal returns flatten while dissipation grows. Past the breakpoints, small perturbations induce **hysteresis**—the system does not seamlessly return to pre-shock performance without explicit buffer rebuilds.

Polarization dynamics. Outputs that are *too* effective at capturing demand can over-concentrate attention and data, starving diversity and resilience elsewhere. This "winner-take-most" pressure increases systemic fragility: a hot feature drives rapid adoption, saturates buffers, and elevates the cost of any correction.

Semantic black holes. In the semantic field view, content and interaction patterns form **attractors**. When outputs become narrowly self-reinforcing, the system's semantic spread collapses into a tight basin—**a semantic black hole**:

Low semantic spread entropy (content diversity collapses),

High collapse readiness (small shocks trigger large regressions),

Rising saturation-stress (buffers and review queues fill).

Inside such basins, behavior appears **near-linear** (highly predictable) but only because diversity has vanished; outside the basin, the system behaves nonlinearly and is hard to recover. Preventing

these states is central to safe scaling: we must detect semantic concentration early and **steer decoding/routing** to re-introduce healthy spread while preserving task value.

1.3 Contributions: theory → metrics → control → governance (closed loop)

This work contributes a practical, self-contained stack that closes the loop from **theory** to **governance**:

(A) Theory — Surplus-aware interaction model.
We formalize each interaction as a micro-production step with explicit surplus S and residual R, coupled across four reservoirs (compute, financial, institutional, attention/cognition). Collapse is a threshold phenomenon driven by low residual and semantic over-concentration.

(B) Metrics — Three early-warning indicators.

SSE (Semantic Spread Entropy): a rolling estimate of content/plan diversity. **Low SSE** flags semantic concentration.

CRP (Collapse Readiness Proxy): a composite of backlog growth, rework ratio, and variance of turnaround—rising **CRP** means small shocks will propagate.

SSI (Saturation–Stress Index): pressure on buffers (queues, review load, context occupancy). **High SSI** signals imminent throttle or quality drift.

These are computed from standard logs (latency, tool use, revisions, embedding angles, format checks) without special infrastructure.

(C) Control — Dissipative decoding objective and micro-MPC.
We modify sampling with a per-step objective

$$J = L - \lambda \Gamma,$$

where L captures task value proxies (likelihood/progress/format integrity) and Γ penalizes predicted dissipation (topic drift, needless tool/mode switches, formatting breaks). A **trust region** (e.g., KL or Δ-logit bounds) ensures stability. The weight λ is **endogenously tuned** by live telemetry:

$$\lambda = f(\mathrm{SSE},\ \mathrm{CRP},\ \mathrm{SSI}),$$

tightening dissipation penalties when spread collapses or buffers saturate. A lightweight **micro-MPC** (event-triggered lookahead over the next tokens/tools) selects actions with best near-term surplus subject to safety caps.

(D) Governance — Routing, buffers, and auditability.
We add policy levers above decoding:

Routing & caps: anti-concentration rules to avoid semantic black holes, minimum-spread constraints for exploration, and per-tenant caps aligned with buffer health.

Buffers & compensation: reserve pools for oversight and data contributors during high-risk phases to keep residuals above thresholds.

Audit: log L, Γ, trust-region stats, and trigger events for transparent review, incident drills, and falsification.

Closed loop. Telemetry feeds indicators (SSE/CRP/SSI) → adjusts decoding (λ, trust region, lookahead) → stabilizes surplus and residual → updates routing and governance → which, in turn, shape telemetry. The result is **surplus-aware AGI control** that scales capability while keeping collapse risk measurable, steerable, and auditable.

2. Surplus Dynamics of AGI Systems

We model real deployments as coupled reservoirs that both **create value** and **consume capacity**. Each interaction updates stocks of residual capacity; when these stocks run thin, the system drifts toward self-reinforcing degradation. This section gives a minimal, self-contained calculus you can instrument from ordinary logs.

2.1 Five surplus flows: material (M), financial (F), institutional (I), attention (A), cognition (C)

Let $k \in \{M, F, I, A, C\}$ index five coupled flows. Each has a **residual stock** $R_k(t)$ (what remains usable at step t):

Material M**:** compute, memory, bandwidth, storage I/O.
Examples: available GPU-min, RAM headroom, queue slots.

Financial F**:** budgetary room to run/scale tasks.
Examples: marginal cost headroom, token/compute spend buffers.

Institutional I**:** organizational throughput for review, security, compliance, escalation.
Examples: reviewer-hours, legal bandwidth, approval windows.

Attention A**:** end-user and operator attention, context window occupancy, notification budgets.
Examples: open tabs, active sessions, notification rate caps.

Cognition C**:** human + agent planning bandwidth (focus depth, working memory, error-correction time).
Examples: allowable retries, edit/rewrite cycles, think-time quotas.

Surplus is **not** a single scalar; it is the multi-reservoir **room to maneuver** that keeps value creation stable.

2.2 Minimal discrete update with generation, capacity, loss, and cross-type conversion

We advance the system in discrete **semantic ticks** (time index t). For each flow k:

$$R_k(t+1) \;=\; \underbrace{\min\Big\{R_k(t) + G_k(t) - L_k(t) - \mu_k R_k(t) + \sum_{j\neq k}\big(C_{j\to k}(t) - C_{k\to j}(t)\big),\; K_k(t)\Big\}}_{\text{capacity-limited update}}$$

Terms (all observable or estimable):

Generation $G_k(t)$**:** value that replenishes residuals in flow k.
Examples: better routing lowers F spend → G_F; tooling that shortens review cycles adds G_I; well-formatted outputs free edit-time → G_C, G_A.

Loss $L_k(t)$**:** direct consumption by the current work (tokens, latency, reviewer minutes, attention minutes).

Leak $\mu_k R_k(t)$**:** background decay (hardware contention, policy churn, attention fatigue).

Cross-type conversion $C_{k\to j}(t)$**:** spending one flow to top up another at an **exchange rate** $\alpha_{k\to j}$.
Examples: paying reviewers ($F \to I$), precomputing embeddings ($M \to C$ to save think-time), notification throttles ($I \to A$ via policy).

Capacity $K_k(t)$**:** hard/soft caps (GPU quota, daily budget, reviewer-hours, inbox threshold).
Note: K_k can itself depend on past load (e.g., staff fatigue lowers tomorrow's K_I).

A compact way to encode conversions is a **flow matrix** $\mathbf{C}(t)$ with entries $C_{k\to j}(t)$ and budget constraints

$$\sum_j C_{k\to j}(t) \;\le\; B_k(t), \quad C_{k\to j}(t) \le \alpha_{k\to j}\, R_k(t),$$

ensuring conversions are feasible and priced.

Link to decoding. The decoder's action (token choice, tool call, formatting) changes G_k, L_k, and the mix $C_{k\to j}$. That is why we later couple decoding to live telemetry of R, K.

2.3 Bistability and hysteresis from S-shaped gain + capacity breakpoints

Even this minimal update admits **multiple operating points**:

S-shaped gain. Many G_k vs. load curves rise fast at first (coordination benefits), then flatten (congestion), and finally turn down (rework dominates). A simple surrogate:

$$G_k(t) \;=\; \gamma_k \,\frac{x_k(t)^{h_k}}{\theta_k^{h_k} + x_k(t)^{h_k}} \;-\; \beta_k\, x_k(t),$$

where x_k is an activity proxy (requests/min, tool invocations), γ_k is peak replenishment, and the final term models dissipation that grows with activity.

Capacity breakpoints. As x_k approaches K_k, queues form; effective service rate drops, inflating L_k and μ_k.

Result: The fixed-point map $R_k \mapsto R_k'$ can intersect the identity at **two stable equilibria** (healthy / degraded) separated by an unstable one. When rising load pushes the system past the middle crossing, it **snaps** to the degraded basin (low R, low K, high L). Reducing load afterward does **not** immediately restore the healthy state—buffers and fatigue must be rebuilt first. This **hysteresis** matches field behavior: "things fall apart quickly and recover slowly."

Engineering takeaway. Avoid running near breakpoints; keep safety margin on x_k / K_k, and use early warnings (next section) to act *before* the saddle-node flip.

2.4 Three operational indicators: SSI, CRP, SSE

These indicators are computable from standard traces (latency, queue length, tool switches, revisions, embeddings, formatting checks). They standardize **when to tighten decoding**, **when to route/cap**, and **when to rebuild buffers**.

(a) SSI — Saturation–Stress Index (capacity pressure)

A unitless 0–1 score aggregating utilization, backlog, and cap hits:

$$\mathrm{SSI}_t \;=\; \sum_k w_k \Bigg[\underbrace{\frac{x_k(t)}{K_k(t)}}_{\text{utilization}} \;+\; u_k \underbrace{\frac{Q_k(t)}{Q_k(t)+K_k(t)}}_{\text{backlog pressure}} \;+\; v_k \underbrace{\mathbb{1}\{x_k(t) \;\geq\; K_k(t)\}}_{\text{cap hits}} \Bigg] \Bigg/ \sum_k w_k,$$

where Q_k is queued work. **High SSI** ⇒ imminent throttling, rising latency, quality drift.

(b) CRP — Collapse Readiness Proxy (fragility)

Captures how quickly a shock would propagate:

$$\mathrm{CRP}_t \;=\; a \max\{0, \Delta Q(t)\} \;+\; b\,\mathrm{ReworkRate}(t) \;+\; c\,\mathrm{CV}[T_{\mathrm{turn}}(t)] \;+\; d\,\mathrm{NearMiss}(t),$$

ΔQ: backlog growth (positive part).

ReworkRate: fraction of outputs triggering edits/returns/escalations.

$\mathrm{CV}[T_{\mathrm{turn}}]$: coefficient of variation of turnaround time.

NearMiss: rate of outputs narrowly passing format/policy checks.
High CRP ⇒ small perturbations will amplify (thin buffers + volatile service).

(c) SSE — Semantic Spread Entropy (diversity)

Measures semantic concentration using clusters/topics/plans derived from embeddings, tool traces, or intent tags. Let $p_i(t)$ be the share of activity in cluster i over a rolling window of length W:

$$\mathrm{SSE}_t \;=\; \frac{-\sum_i p_i(t) \log p_i(t)}{\log N},$$

normalized to $[0, 1]$ with N clusters. **Low SSE** ⇒ content/plan diversity is collapsing toward a narrow attractor (risk of "semantic black hole").

Putting them together

Green: $\mathrm{SSI} < 0.5,\ \mathrm{CRP} < \tau_{\mathrm{crp}},\ \mathrm{SSE} > 0.6$

Caution: $\mathrm{SSI} \in [0.5, 0.7]$ **or** $\mathrm{CRP} \uparrow$ **or** $\mathrm{SSE} \downarrow$

Action: $\mathrm{SSI} > 0.7$ **or** $\mathrm{CRP} > \tau_{\mathrm{crp}}$ **or** $\mathrm{SSE} < 0.4$

Control hooks (used in §4 and beyond):

If **SSI rises**, tighten trust regions, prefer low-cost tool paths, reduce branching.

If **CRP rises**, slow intake (routing/caps), increase buffer rebuild (rest/rehearsal), escalate human-in-the-loop.

If **SSE falls**, inject exploration quotas, diversify prompts/tools, and deconcentrate routing to avoid semantic black holes.

This trio is intentionally **minimal**: fast to compute, easy to audit, and sufficiently predictive to steer decoding and policy before the system crosses dangerous thresholds.

3. Semantic Field Geometry (Self-Contained Primer)

This section introduces a compact, implementation-ready geometry for reasoning about how meanings **evolve, concentrate, or drift** during generation. It requires no external theory: we define the variables, show why "semantic black holes" look near-linear from the inside, derive useful engineering quantities, and map everything to concrete logs (logits, embeddings, and stepwise traces).

3.1 Variables: semantic tension iT, orientation θ, tick-time τ, projection operator $\hat{O}$

Let $e_t \in \mathbb{R}^d$ be the embedding of the current content window at **semantic tick** t (a tick is one controllable step: e.g., 10–40 tokens, one tool call, or one function return). Let $g \in \mathbb{R}^d$ be a **goal vector** distilled from the task brief (instruction summary, schema, style guide).

(a) Orientation θ_t.
The angular alignment between what is being produced and what the task asks for:

$$\cos\theta_t \;=\; \frac{\langle e_t,\, g\rangle}{\|e_t\|\;\|g\|}, \qquad \theta_t \in [0, \pi].$$

Small θ_t ⇒ on-track; large θ_t ⇒ off-track or drifting.

(b) Projection operator $\hat{0}$.

A linear operator that **keeps** components that lie in the **constraint subspace** S (format, policy, persona, ontology) and discards the rest. In practice, build S from a basis $B = \{s_i\}$ (e.g., average embeddings of required sections, examples, schema tags):

$$\hat{\mathbf{0}}(e) \;=\; \sum_i \frac{\langle e,\, s_i \rangle}{\|s_i\|^2}\, s_i\,.$$

Two useful scalars:

$$\phi_t \;=\; \frac{\|\hat{\mathbf{0}}(e_t)\|}{\|e_t\|} \quad \text{(in-subspace proportion)}, \qquad \ell_t \;=\; \sqrt{1-\phi_t^2} \quad \text{(orthogonal leakage)}.$$

(c) Semantic tension $iT_t \geq 0$.
A **stored-tension** scalar that rises when the current state is (i) misaligned with the goal or (ii) leaking orthogonally to constraints or (iii) over-concentrated. A practical composite:

$$iT_t \;=\; \alpha\,(1-\cos\theta_t) \;+\; \beta\,\ell_t \;+\; \gamma\,(1-\mathrm{SSE}_t),$$

where SSE_t is the semantic spread entropy from §2.4 (normalized $[0,1]$). Larger iT_t means "pressure to correct."

(d) Tick-time τ.
A counter for **control granularity**. Choose $\Delta\tau$ so each tick captures one meaningful planning stride: sentence/phrase window, tool decision, or function hop. Derivatives like $\dot{\theta}_t \approx \Delta\theta\,/\Delta\tau$ are then well-behaved.

3.2 Attractors and "near-linearity" inside semantic black holes

Define a **potential** $U(e)$ that is low where the system tends to settle (recurring topics, styles, plans). **Attractors** are basins of low U with positive curvature (you get "pulled back" when perturbed). When output begins to **self-reinforce** (users click the same thing, routing favors a path, prompts echo prior phrasing), the basin deepens and **spread collapses**.

From inside such a basin—our **semantic black hole**—three empirical signatures appear:

Low spread: SSE_t drops; activity concentrates in a few clusters.

Stiff alignment: θ_t and ϕ_t fluctuate little; the Jacobian of decoding w.r.t. prompts/tools is small.

Near-linearity: locally, outputs respond like a **linear system** to small nudges; the same nudge yields nearly the same change each time.

Near-linearity is not "good" or "bad"—it is a **warning** that diversity has vanished. It enables gentle linear control **temporarily**, but recovery to a healthy regime requires **re-introducing spread** (raise SSE) and lowering stored tension iT.

3.3 Derived quantities: semantic mass, force, and energy (engineering interpretations)

We now define three handy analogues that turn logs into **actionable control signals**.

(a) Semantic mass m_s (inertia / stickiness).
How resistant the content trajectory is to steering. Estimate by relating a control **impulse** I_t to the **angular acceleration** it causes:

$$m_s(t) \;\approx\; \frac{I_t}{\Delta^2 \theta_t \,/\Delta \tau^2}.$$

I_t can be the **trust-region nudge** size (e.g., per-tick KL from baseline, Δ-logit norm) or the magnitude of a structured **steering token**/tool call.

High m_s: the narrative is "heavy"—hard to redirect (lock-in).

Low m_s: easy to steer—good for exploration, risky for drift.

(b) Semantic force F_s (pull back toward/away from goal).
Define a simple **goal-mismatch potential** around θ:

$$U(\theta) \;=\; \tfrac{1}{2}\,\kappa\big(1-\cos\theta\big)^2, \qquad F_s \;=\; -\frac{\partial U}{\partial \theta} \;=\; -\kappa\big(1-\cos\theta\big)\sin\theta\,.$$

If θ is moderate, $F_s \approx -\kappa\,\theta$ (restoring).

Calibrate κ from replay so "1° misalignment" maps to a chosen corrective strength.

(c) Semantic energy $E_s = T_s + U$ (how "hot" the run is).
With angular rate $\omega_t = \Delta\theta\,/\Delta\tau$:

$$T_s \;=\; \tfrac{1}{2}\,m_s\,\omega_t^2, \qquad E_s \;=\; T_s \;+\; U(\theta_t).$$

High T_s: flailing—direction changing rapidly (likely dissipation).

High U: stably off-goal—needs a reset or stronger projection.

Use E_s as an **interrupt** signal: if E_s spikes while SSI/CRP are rising, slow intake, escalate oversight, or trigger an exploration/diversification routine.

Why these help. Controllers reason better with **rates** and **curvatures** than with raw angles. m_s, F_s, E_s summarize "how hard to push," "which way," and "whether to back off."

3.4 Mapping to observables: logits concentration, embedding angles, stepwise estimates

Everything above is directly computable from standard traces.

Inputs you already log

Logits / token probabilities at each step.

Embeddings for rolling content windows (sentence/phrase level).

Tool/mode events (calls, returns), latency, retries.

Practical estimators (per tick t)

Logit concentration (narrowness).

Entropy $H_t = -\sum_v p_t(v)\log p_t(v)$.

Normalized concentration $C_t^{\text{logit}} = 1 - H_t / \log |V|$.

Use: high C^{logit} + rising iT ⇒ "confident but wrong" risk; tighten trust region, increase projection $\hat{\mathbf{0}}$.

Embedding orientation & drift.

Compute θ_t via cosine with the goal vector g.

Rates: $\omega_t \approx \Delta\theta / \Delta\tau$, acceleration $\approx \Delta^2\theta / \Delta\tau^2$.

Use: derive m_s, F_s, E_s. Large m_s ⇒ apply **structured** steering (section headers, schema tags) rather than soft token bias.

Projection compliance & leakage.

Maintain a constraint basis $B = \{s_i\}$ (format exemplars, policy anchors).

Compute ϕ_t, ℓ_t from $\hat{\mathbf{0}}(e_t)$.

Use: high ℓ_t increases iT; penalize Γ (dissipation) for off-subspace moves; prefer tool paths that restore ϕ_t.

Semantic spread entropy (SSE).

Cluster window embeddings over a rolling horizon W; let p_i be cluster shares.

$\text{SSE}_t = -\sum_i p_i \log p_i / \log N$.

Use: low SSE flags **semantic black hole** formation; inject exploration (alternative plans, paraphrase prompts, diversified routing).

Stepwise energy gate.

Compute $E_s(t)$. If E_s exceeds a learned threshold while **SSI/CRP** are elevated, trigger **micro-MPC**: short lookahead over next tokens/tools to pick the minimal-dissipation branch that still improves U.

Recommended defaults

Ticking ($\Delta\tau$): 20 tokens or 1 tool decision; adaptively align to punctuation/AST nodes.

Goal vector g: average of instruction, section headers, schema names; refresh at section boundaries.

Constraint basis B: 10–30 exemplars (format/policy), updated nightly from high-quality runs.

Calibration: fit κ and energy thresholds on replay; validate that "Action" alarms (from §2.4)

precede quality regressions by your target lead time.

What you get: a **closed mathematical loop** that stays entirely within ordinary telemetry. iT, θ, τ, $\hat{\mathbf{0}}$ are simple to compute; m_s, F_s, E_s are cheap derived signals that tell you **when to nudge, how hard, and when to diversify**—before the system slides into brittle, collapse-prone regimes.

4. Dissipative Decoding: The L–Γ Objective

We modify decoding so each controllable step maximizes **task value** while minimizing **predicted dissipation**, under a stability constraint. The controller acts at the same granularity as §3's **semantic tick** (e.g., 10–40 tokens or one tool decision), uses only ordinary telemetry, and is fully auditable.

4.1 Per-step objective $J = L - \lambda\,\Gamma$: value vs. dissipation

At tick t, for a candidate action u (next token, tool call, plan edge), define

$$\boxed{J_t(u) \;=\; L_t(u) \;-\; \lambda_t\,\Gamma_t(u)}$$

$L_t(u)$: **value** proxy—how much the action advances the task safely and efficiently.

$\Gamma_t(u)$: **dissipation** proxy—the downstream cost we are likely to incur (drift, flip-flopping modes, format breakage).

$\lambda_t \;\geq\; 0$: trade-off weight, **adapted online** from system health (SSI/CRP/SSE; §4.5).

Subject to a **trust region** against the baseline logits p_t (stability; §4.4).

4.2 Value term L: likelihood, progress, structure risk, latency, length (auditable)

We write L as a **sum of auditable components**, each normalized to $[0, 1]$ or z-scored over a rolling window:

$$L_t(u) \;=\; \underbrace{\alpha_1\,\Delta\log p_t(u)}_{\text{likelihood gain}} \;+\; \underbrace{\alpha_2\,\mathrm{Prog}_t(u)}_{\text{task progress}} \;-\; \underbrace{\alpha_3\,\mathrm{StructRisk}_t(u)}_{\text{schema/format risk}} \;-\; \underbrace{\alpha_4\,\mathrm{Latency}_t(u)}_{\text{runtime cost}} \;-\; \underbrace{\alpha_5\,\mathrm{Length}_t(u)}_{\text{verbosity cost}}.$$

Recommended, measurable ingredients

Likelihood gain $\Delta\log p$**:** log-prob under the base model (or improved under reranker) relative to local average. Favor fluent, well-supported continuations.

Task progress Prog**:** a bounded scalar from §3 signals:
$\mathrm{Prog}_t = w_\theta\,(\cos\theta_{t+1} - \cos\theta_t) \;+\; w_\phi\,(\phi_{t+1} - \phi_t)$,
rewarding better **goal alignment** and **projection compliance**.

Structural risk StructRisk**:** probability (from a light classifier/parser) that the action leads to a

schema break (e.g., JSON invalid, section missing, citation fail).

Latency & length: predicted incremental wall-time tokens/tool runtime, and token budget consumption.

Auditing. Log each α_i and the raw sub-scores per step. This lets reviewers answer "*why did we prefer this path?*" with numbers.

4.3 Dissipation term Γ: topic drift, mode/tool switching, format integrity

$$\Gamma_t(u) \;=\; \underbrace{\beta_1\,\mathrm{Drift}_t(u)}_{\text{topic/intent drift}} \;+\; \underbrace{\beta_2\,\mathrm{ModeFlip}_t(u)}_{\text{tool/mode chattering}} \;+\; \underbrace{\beta_3\,\mathrm{FmtDebt}_t(u)}_{\text{format integrity debt}} \;+\; \underbrace{\beta_4\,iT_{t+1}}_{\text{stored tension carryover}} \quad .$$

Concrete, cheap estimators

Topic/intent drift Drift**:** predicted increase in misalignment angle θ or orthogonal leakage ℓ (from §3) over a one-tick lookahead.
Example: $\mathrm{Drift}_t = \max(0, \theta_{t+1} - \theta_t) + \lambda_\ell \max(0, \ell_{t+1} - \ell_t)$.

Mode/tool chattering ModeFlip**:** penalty for **rapid switching** (e.g., tool A → B → A in short windows). Estimate with an exponential moving count of transitions or an ℓ_0-like cost on mode changes.

Format integrity debt FmtDebt**:** cumulative probability that the current branch will **require repair** to meet format/policy (distinct from StructRisk, which is immediate). Maintain a running debt meter that increases with borderline tokens and decreases with corrective structure tokens.

Stored tension iT**:** directly from §3.1; high iT means unaddressed misalignment or over-concentration that will cause **downstream rework**.

Why split risk vs. debt?

StructRisk belongs in L as an immediate value deduction (it hurts the *current* step's value).

FmtDebt belongs in Γ as a **future burden** we are accumulating.

4.4 Event-triggered micro lookahead (micro-MPC) under KL/Δlogit trust regions

When signals suggest rising risk, run a **short, cheap controller** over the next H micro-steps (e.g., 3–10 tokens or 1 tool hop):

Trust region (stability):
Keep the adjusted distribution q_t close to baseline p_t via

$$\mathrm{KL}(q_t \,\|\, p_t) \;\le\; \varepsilon_t \quad \text{or} \quad \|\,\Delta\mathrm{logits}_t\,\|_2 \;\le\; \delta_t,$$

with ε_t, δ_t auto-scaled by SSI/CRP (tighten under stress).

When to trigger micro-MPC

E_s spike (semantic energy; §3.3), or

SSI $> \tau_{\text{ssi}}$, CRP $> \tau_{\text{crp}}$, or SSE $< \tau_{\text{sse}}$, or

Near a schema boundary (section close, code block end, JSON object close).

What it does (pseudocode)

```
candidates = generate_topK(p_t) ∪ {tool_calls}
for each candidate u:
    rollout H-steps with light heuristics (no full beam)
    compute L̄ = Σh γ^h L_{t+h}(u_h)
    compute Γ̄ = Σh γ^h Γ_{t+h}(u_h)
    J̄ = L̄ - λ_t Γ̄
choose argmax J̄ subject to KL/Δlogit bounds; else fallback to baseline
```

$\gamma \in (0, 1]$ is a short-horizon discount (e.g., 0.8–0.95).

Use cached embeddings and a light parser to keep cost small.

Fallback if trust region saturates or J variance is high.

Outcome. Micro-MPC quietly avoids high-dissipation branches (e.g., a flashy phrase that derails structure, or a tool flip that will be undone).

4.5 Coupling λ to system telemetry: $\lambda = f(\text{SSI}, \text{CRP}, \text{SSE})$

$$\boxed{\lambda_t \;=\; \text{clip}\Big(\lambda_{\min},\; \lambda_{\max},\; \lambda_0 \left[1 + a\,\text{SSI}_t + b\,\text{CRP}_t + c\,(1 - \text{SSE}_t)\right]\Big)}$$

Interpretation

When **capacity pressure** (SSI) or **fragility** (CRP) rises, the controller **pays more attention** to dissipation (Γ).

When **spread collapses** (SSE ↓), we also increase λ to counter semantic black-hole formation.

Reasonable defaults

$\lambda_0 \in [0.2, 0.6]$, $\lambda_{\min} = 0$, $\lambda_{\max} \in [1.5, 3]$.

$a \in [0.5, 1.0]$, $b \in [0.5, 1.2]$, $c \in [0.3, 0.8]$.

Smooth λ_t with EMA (e.g., 0.7) to avoid control chatter.

Safety couplings (recommended)

If SSI > 0.7, also **tighten trust region** ($\varepsilon_t \downarrow$) and **raise cost weights** α_4, α_5 (latency/length).

If CRP $> \tau_{\text{crp}}$, **throttle intake / route away** and **increase human-in-the-loop**.

If SSE $<$ 0.4, enforce **minimum exploration quotas** (diversify prompts/tools) until SSE recovers.

Implementation checklist (one page)

Compute per-tick L and Γ with logged features (keep units and normalizations in metadata).

Log λ_t, ε_t/δ_t, chosen action, and a breakdown of L and Γ sub-terms.

Trigger micro-MPC on E_s spikes or SSI/CRP/SSE thresholds; store candidate J table for audit.

Nightly calibration: scale α_i, β_i so typical magnitudes of L and Γ are comparable; target $50:50$ when healthy.

Red-team test: synthetic scenarios that force tool chattering, schema edge cases, and drift; verify the controller chooses low-Γ branches and that alarms precede regressions by the desired lead time.

This L–Γ controller converts ordinary decoding into a **surplus-aware** optimization that is stable, transparent, and tunable—steering generation away from high-dissipation branches precisely when the system is most vulnerable.

5. Stack Architecture (Four Layers, One Loop)

The system is a single sensing–deciding–acting loop that runs across four layers. **L1** maintains the semantic map and constraints; **L2** performs dissipative decoding with the $J = L - \lambda\,\Gamma$ controller; **L3** turns raw traces into health signals (SSI/CRP/SSE, plus E_s); **L4** sets routing, caps, and audit policy. Data flows clockwise; control levers flow back down to keep surplus positive and collapse risk low.

L1 — Semantic Terrain (state and constraints)

Purpose. Keep a live, minimal state that the controller can steer against.

Core objects

Goal vector g from brief/instructions.

Constraint subspace S with basis $B = \{s_i\}$ (format/policy/persona exemplars); projection operator $\hat{\mathbf{0}}$.

Rolling embeddings e_t for the current window; **clusters** for spread estimation (SSE).

Tick scheduler (decides when a "semantic tick" occurs: ~20 tokens, or at tool/section boundaries).

Outputs to L2/L3

Alignment & leakage: $\theta_t,\ \phi_t,\ \ell_t$.

Tension & dynamics: $iT_t,\ m_s,\ F_s,\ E_s$ (from §3).

Spread: cluster IDs and SSE_t.

Inputs from L4

Required schemas, redlines (disallowed content/tools), persona/style packs, privacy constraints.

L2 — Decoding Control (the $J = L - \lambda\,\Gamma$ engine)

Purpose. Choose the next token/tool/plan edge that maximizes near-term value while minimizing dissipation, under a stability bound.

Mechanics

Objective: $J_t(u) = L_t(u) - \lambda_t\,\Gamma_t(u)$.

Trust region: KL or ‖Δlogits‖ bounds vs. baseline logits p_t.

Micro-MPC: short lookahead (3–10 steps) when alarms fire or at schema edges.

Action gates: structure keepers (headers, JSON delimiters), tool budgeters, stop/abort sequences.

Router hook: if a tool/agent path wins J, dispatch with rate limits attached (from L4).

Outputs to L3

Chosen action, candidate table $\{\overline{J}, \overline{L}, \overline{\Gamma}\}$, realized costs (latency, tokens), trust-region stats, tool/mode transitions.

Inputs from L1/L3/L4

From **L1:** $g, B, \theta, \phi, \ell, iT, m_s, E_s$.

From **L3:** λ_t, current trust-region budgets, health state.

From **L4:** routing/caps, policy toggles (e.g., exploration quota on).

L3 — Telemetry & Early-Warning (health and adaptation)

Purpose. Convert traces into **operational indicators** and adapt controller tightness.

Signals (rolling windows)

SSI (saturation–stress): utilization, backlog pressure, cap hits.

CRP (collapse readiness): backlog growth, rework rate, turnaround variance, near-misses.

SSE (semantic spread entropy): normalized topic/plan diversity.

Energy E_s: from §3 (flailing/off-goal detection).

Logic

Compute health state: *Green / Caution / Action*.

Derive controller settings: $\lambda_t = f(\text{SSI,CRP,SSE})$; tighten/loosen ε_t (KL) and δ_t (Δlogit).

Raise **BH-1/BH-2** alerts (semantic black-hole risk) when SSE is low and CRP/SSI are high.

Outputs to L2/L4

Real-time λ_t, ε_t, δ_t, trigger flags for micro-MPC.

Aggregated SLO dashboards, alarms, and incident timelines.

L4 — Policy & Governance (routing, caps, compensation, audit)

Purpose. Shape demand and incentives so the system stays inside safe envelopes and remains auditable.

Controls

Routing & caps (per tenant/feature/region): request QPS, token budgets, tool-call/min, concurrent jobs, exploration quota.

Anti-concentration rules: minimum spread across clusters/skills/providers to prevent semantic black holes.

Buffer management: reviewer hours, escalation queues, rest windows; reserve funds/time when CRP is high.

Compensation & consent: data usage accounting, opt-outs, federated contribution credits.

Audit & disclosure: log policies, controller parameters, trigger events, and rollbacks; publish post-incident notes.

Outputs to L1/L2/L3

Schema/policy packs (L1), hard caps and routing tables (L2), SLO targets & thresholds (L3).

Data Flows (contracts you can implement)

Per-tick event (from L2 to L3)

```
{
  "t": "...", "session": "...", "tenant": "...",
  "logits_hash": "...", "kl_from_base": 0.012, "dlogits_l2": 0.9,
  "action": {"type": "token|tool|plan", "name": "…", "args_hash": "..."},
  "L": {"dlp": 0.34, "progress": 0.18, "structRisk": 0.03, "lat": 120, "len":
18},
  "Gamma": {"drift": 0.05, "modeFlip": 0, "fmtDebt": 0.02, "iT": 0.21},
  "lambda": 0.72, "micro_mpc": true, "candidates": 6,
  "lat_ms": 140, "tokens": 22, "tool_ms": 0
```

```
}
```

Health snapshot (from L3 to L2/L4)

```
{
  "window": "t-120s..t",
  "SSI": 0.63, "CRP": 0.41, "SSE": 0.38,
  "Es_spike": true, "state": "Action",
  "lambda": 0.72, "kl_budget": 0.008, "dlogits_budget": 0.7
}
```

Policy packet (from L4 to L2/L1)

```
{
  "caps": {"qps": 50, "tok_min": 0, "tok_max": 2e5, "tool_per_min": 80},
  "routing": {"tenantA": {"providers": ["P1","P2"], "min_spread": 0.55}},
  "exploration_quota": {"min_SSE": 0.5, "quota_pct": 0.1},
  "schemas": ["spec.v3.json", "persona.alpha.yaml"],
  "privacy": {"pii_redact": true, "trace_retention_days": 30}
}
```

Caps (where to place them) and how they adapt

Map caps to the five reservoirs (§2.1):

Material (M): tokens/sec, concurrent jobs, VRAM/CPU quotas, tool runtime ceilings.

Financial (F): daily spend, marginal cost per request, surge price floors (optional).

Institutional (I): reviewer-hours/day, max escalations/hour, policy-check QPS.

Attention (A): notification rate, active session count, max context reuse.

Cognition (C): retries per task, edit passes, allowed branching width.

Adaptive rules

If **SSI**↑: automatically lower KL budgets, reduce branching width, prefer low-latency tools.

If **CRP**↑: cap intake QPS, raise human-in-the-loop, lengthen rest/rehearsal windows.

If **SSE**↓: enforce exploration quota and deconcentrate routing.

Rollback Paths (fail safely, not silently)

R0 Soft nudge: increase λ, tighten trust region, enable micro-MPC.

R1 Structural guardrails: force schema tokens, freeze tool switching for H ticks.

R2 Route & throttle: shift traffic to cooler shards/providers; reduce QPS.

R3 Degrade gracefully: turn off expensive tools; switch to template/FAQ answers where safe.

R4 Human takeover: escalate to reviewer; pause session continuation.

R5 Suspend feature/tenant: if BH-2 persists (low SSE + high CRP/SSI), halt and rebuild buffers.

R6 Full rollback: revert controller coefficients, policy packs, or last release; restore from last "Green" snapshot.

Each rollback action is **audited** (who/why/when, associated metrics before/after) and tied to explicit **exit criteria** (e.g., SSE > 0.55 for 30 minutes, CRP below threshold).

Putting the loop together

Sense (L3): compute SSI/CRP/SSE and E_s from L2/L1 traces.

Decide (L3→L2/L4): set λ, trust-region budgets, raise alerts; L4 adjusts caps/routing.

Act (L2): decode with $J = L - \lambda\,\Gamma$, run micro-MPC when required.

Audit & Learn (L4/L3): store breakdowns, compare against SLOs, calibrate coefficients nightly.

This four-layer, one-loop stack is small enough to bolt onto existing systems, yet complete enough to **stabilize surplus, prevent semantic black holes, and provide clean rollback paths** when the environment shifts.

6. Risk Sensing & Early-Warning

We convert the three indicators (§2.4)—**SSE** (semantic spread), **CRP** (collapse readiness), and **SSI** (saturation–stress)—into **crisp alerts** that buy lead time before quality and latency regress. The design goal is simple: **catch the slide while there is still surplus to steer**.

6.1 Composite risk Ξ

Because **low** SSE is risky while **high** CRP/SSI are risky, we combine them with aligned signs. Let

$$\boxed{\Xi_t \;=\; a\,(1-\mathrm{SSE}_t) \;+\; b\,\widetilde{\mathrm{CRP}}_t \;+\; c\,\widetilde{\mathrm{SSI}}_t^{\star}}$$

$1-\mathrm{SSE}_t \in [0,1]$ rises as semantic spread **collapses**.

$\widetilde{\mathrm{CRP}}_t$, $\widetilde{\mathrm{SSI}}_t^{\star}$ are normalized (z-score or min–max) so terms are comparable.

$\mathrm{SSI}_t^{\star}$ is a **conservative stress proxy**:

$$\mathrm{SSI}_t^{\star} \;=\; \max_k \mathrm{SSI}_k(t) \quad \text{(max across reservoirs/shards)},$$

or the 95th percentile across shards if you prefer robustness.

Optional surge sensitivity. Add slope terms to catch fast onsets:

$$\Xi_t^{\text{surge}} \;=\; \Xi_t \;+\; d\left[\Delta^+(1-\text{SSE}_t)\right] \;+\; e\left[\Delta^+ \widetilde{\text{CRP}}_t\right],$$

where $\Delta^+ x = \max(0, x_t - x_{t-1})$.

Operating bands (example):

Green: $\Xi < 0.35$

Caution: $0.35 \le \Xi < 0.55$

Action: $0.55 \le \Xi < 0.75$

Emergency: $\Xi \ge 0.75$

(Choose bands so **false positives** are acceptable; misses are costlier.)

6.2 Black-hole alerts: BH-1 and BH-2

We define two alert levels aimed at **semantic black hole** prevention (concentration + fragility).

BH-1 (Early) — "Concentration + Fragility"

Trigger when **both** are true over a short rolling window W_s (e.g., 60–180 s):

$$\text{SSE}_t \le \tau_{\text{sse}} \quad \wedge \quad \widetilde{\text{CRP}}_t \ge \tau_{\text{crp}}\,.$$

Typical defaults: $\tau_{\text{sse}} = 0.45$, $\tau_{\text{crp}} = \,+0.8\sigma$ above baseline.

Automations on BH-1:

L2: raise λ (dissipation weight), tighten KL/Δlogit budgets, enable micro-MPC.

L2/L1: inject **exploration quota** (diversify plans/tools), enforce schema tokens at boundaries.

L4: deconcentrate routing (spread traffic across clusters/shards), apply soft caps on hot tenants.

BH-2 (Severe) — "Composite risk breach"

Trigger when the **composite** exceeds a calibrated threshold for a medium window W_m (e.g., 5–15 min):

$$\Xi_t \ge \tau_{\Xi} \quad \text{(sustained over } W_m \text{ or repeated } n \text{ times in } W_m\text{)}.$$

Typical defaults: $\tau_{\Xi} = 0.65$, $n = 3$.

Automations on BH-2:

Escalate rollback path **R2→R5** (§5): route/throttle, degrade gracefully, add human-in-the-loop, or suspend the hot feature/tenant.

Reserve/rebuild buffers (reviewer hours, rest windows); log a **notifiable incident** with controller traces (L, Γ, λ, KL).

Why two alerts? BH-1 is a **fast local detector** (cheap & twitchy); BH-2 is a **confirmatory**

composite (slower but high precision). Together they provide **early nudges without whiplash**, and **credible triggers** for costly mitigations.

6.3 Calibration: windows, thresholds, and lead-time targets

Windows

Use **three horizons** to balance reactivity and stability:

W_s **short** (30–180 s): for BH-1 and surge terms.

W_m **medium** (5–15 min): for BH-2 and control adaptation (λ, KL budgets).

W_l **long** (1–6 h): for drift baselines and shift detection (seasonality, releases).

Smooth with EMAs (decay 0.6–0.85); **debounce** alerts (require k consecutive breaches or duty-cycle ≥ 40% of the window).

Thresholds (data-driven)

Collect 2–4 weeks of traces with labels for "quality dip," "latency spike," "incident."

Normalize CRP/SSI by z-score within tenant/feature; keep SSE [0, 1].

Fit $\tau_{\text{sse}}, \tau_{\text{crp}}, \tau_{\Xi}$ to maximize **lead time @ 80–90% precision** (survival or time-to-event analysis).

Backtest BH-1→action and BH-2→rollback outcomes; tune costs for false positives vs. misses.

Set different bands by environment:

Interactive UX: aim for **2–10 min** BH-1 lead, **10–30 min** BH-2 lead.

Batch/agentic runs: aim for **15–60 min** BH-2 lead; allow stricter thresholds.

Lead-time targets (practical)

BH-1: enough time to **steer decoding** (raise λ, micro-MPC, diversify) **without user-visible degradation**.

BH-2: enough time to **route/throttle and rebuild buffers** before queues harden (avoid hysteresis lock-in).

Pseudocode (reference)

```
# inputs: SSE(t), CRP(t), {SSI_k(t)}
SSE = roll_mean(SSE, W)              # with EMA smoothing
CRPz = zscore(CRP, W_baseline)
SSI_star = max_k(SSI_k)              # or pct95 across shards
SSI_star_z = zscore(SSI_star, W_baseline)
```

```
Xi = a*(1 - SSE) + b*CRPz + c*SSI_star_z
Xi = Xi + d*pos_diff(1 - SSE) + e*pos_diff(CRPz)   # optional surge

BH1 = (SSE <= tau_sse) and (CRPz >= tau_crp) over Ws with debounce k1
BH2 = (Xi  >= tau_Xi)  sustained over Wm or repeats >= n

if BH1:  apply_L2_nudges(); diversify(); soft_deconcentrate()
if BH2:  escalate_R2_to_R5(); rebuild_buffers(); log_incident()
```

Guardrails

Fail-open vs fail-shut: prefer **early conservative** actions (soft caps, exploration) rather than late heavy rollbacks.

Tenant-aware baselines: thresholds per tenant/feature prevent a single dominant stream from hiding localized stress.

Post-incident re-tune: after each BH-2, recompute bands so the **next** event triggers **earlier** for similar patterns.

Outcome. With Ξ + BH-1/BH-2, the system gets a **graduated, auditable early-warning stack**: quick nudges to avoid concentration traps, and decisive triggers to protect buffers—delivering predictable **lead time** before collapse dynamics take over.

7. Experiments and Falsification Plan

We propose three experiments (E1–E3) to stress the system, probe causal levers, and define **explicit falsifiers**. Each experiment can run (a) **replay** on historical traces, (b) **shadow** online (no user impact), and (c) **canary** with guarded exposure. All metrics and triggers rely only on signals defined earlier (SSI, CRP, SSE, E_s, $J = L - \lambda\,\Gamma$, KL/Δlogit).

7.1 E1 — Gain Thresholds ⇒ Incidents and Hysteresis

Purpose. Verify that increasing activity drives the system through **breakpoints** (S-shaped gain + capacity limits), producing (i) a sharp rise in incidents and (ii) **hysteresis** (slower recovery).

Design.

Manipulation: Ramp **request rate** and **branching width** in steps (e.g., +10% every 15 minutes) until just past BH-2. Then ramp down symmetrically.

Hold fixed: model weights, policy packs, routing; no buffer changes.

Environments: one interactive workload and one agentic/batch workload.

Primary outcomes.

Critical load $x^{\backslash *}$ where BH-2 first sustains for W_m.

Incident rate (schema breaks, escalations) vs. load.

Hysteresis gap $\Delta x = x^{\backslash *}_{\downarrow} - x^{\backslash *}_{\uparrow}$ (recovery point minus collapse point).

Expected pattern (confirmatory).

Incident rate stays low until $x \to x^{\backslash *}$, then rises superlinearly.

On ramp-down, health recovers only below $x^{\backslash *}_{\downarrow} < x^{\backslash *}_{\uparrow}$ (nonzero Δx).

Falsifiers.

F1: No superlinear rise in incidents near BH-2.

F2: $\Delta x \approx 0$ across ≥3 runs after adequate cooling time (no hysteresis).

F3: SSI/CRP/SSE bands do **not** lead incident spikes (lead time < 2 minutes for interactive or < 10 minutes for batch).

Replay shadow pseudocode.

```
for load in up_ramp + down_ramp:
    apply_load(load)              # shadow routing
    log(SSI, CRP, SSE, Es, incidents, J, KL)
    if sustained_BH2(): mark_x_star(load)
compute_hysteresis_gap()
```

7.2 E2 — Buffering Shifts Thresholds Right

Purpose. Show that **buffers** (human-in-the-loop, rehearsal, knowledge sharing) shift the collapse threshold $x^{\backslash *}$ **to the right** and reduce fragility (CRP).

Interventions (toggle one at a time).

HITL: add reviewer minutes per hour; priority queue for near-miss outputs.

Rehearsal: daily drills on common failure patterns; update constraint basis B.

Sharing: cache & reuse validated plans/schemas; reduce cognitive rework.

Design.

Repeat E1 ramps under each intervention at matched traffic.

Keep model, policies, and routing identical; only buffer knobs change.

Primary outcomes.

Shift in $x^{\backslash *}$**:** $\Delta x^{\backslash *} = x^{\backslash *}_{\text{with buffer}} - x^{\backslash *}_{\text{baseline}} > 0$.

CRP reduction: area-under-curve of CRP vs. load decreases.

Post-shock recovery time to exit BH-2 shortens.

Secondary outcomes.

Lower rework rate; higher SSE at equivalent load.

Falsifiers.

F4: $\Delta x^{\backslash *} \leq 0$ after applying buffers of meaningful size (≥20% more reviewer-hours or ≥15% rehearsal coverage).

F5: No CRP drop or recovery-time improvement (Wilcoxon p ≥ 0.05 across ≥5 matched days).

Analysis note. Use difference-in-differences vs. a matched control shard; normalize by diurnal seasonality.

7.3 E3 — Multi-Agent Coordination with Routing Costs ("Semantic Currency")

Purpose. Test whether adding **routing costs** for handoffs (a "semantic currency") reduces **mode chattering**, raises SSE, and improves stability.

Setup.

A graph of agents/tools $A_1, \ldots, A_n$ with edges (handoffs).

Assign each edge a **routing cost** $\rho_{i \to j}$ (latency risk, integration overhead).

Modify L2 decision: when candidate u triggers a handoff $i \to j$, include $\rho_{i \to j}$ in Γ.

Treatments.

T0: no routing cost (baseline).

T1: static costs ρ from historical averages.

T2: adaptive costs $\rho_t = \rho_0 \cdot g(\text{SSI}, \text{CRP})$ (costs rise under stress).

T3: add **spread bonus**: small L boost when the path improves SSE without raising CRP.

Primary outcomes.

Mode-flip rate ↓ (transitions/min).

FmtDebt and total Γ ↓ at matched L.

SSE ↑ at equivalent throughput.

Incident rate ↓ and BH-1 frequency ↓.

Falsifiers.

F6: No reduction in mode-flip rate or Γ at matched *L*.

F7: SSE does not improve or incidents do not drop across T1–T3 vs. T0 (A/B $p \geq 0.05$).

F8: Adaptive costs degrade *L* with no compensating drop in CRP/SSI (Pareto dominated).

Implementation sketch.

```
Gamma_t = ... + beta2*ModeFlip + beta3*FmtDebt + sum(rho_edge for handoff in
path)
if stress_high: rho_edge *= k   # adaptive multiplier
J = L - lambda * Gamma
choose argmax_J under trust region
```

7.4 Metrics, Statistics, Survival Analysis, and Explicit Falsifiers

Core metrics.

Quality/SLO: pass rate, schema validity, human redlines, defect density.

Cost/latency: tokens, tool ms, wall time.

Health: SSI, CRP, SSE, E_s, BH-1/BH-2 counts, lead time.

Control internals: L, Γ, λ, KL/Δlogit, candidate J table.

Stability: mode-flip rate, FmtDebt, rework rate, near-misses.

Statistics.

A/B on canaries with hierarchical models (tenant/random effects).

Time-series: Newey–West or state-space models for autocorrelation.

Survival / time-to-event:

Event = BH-2 or incident.

Kaplan–Meier curves, log-rank tests across treatments (E1–E3).

Cox models with covariates (load, caps, buffer minutes, exploration quota).

Target: ≥10–30 min median lead time (batch) or ≥3–8 min (interactive) between BH-1 and incident.

Effect sizes: report Δ in $x^{\backslash *}$, hazard ratios (HR), and partial R^2.

Powering: simulate from replay to size canaries (e.g., detect 20% HR reduction with 80% power @ α=0.05).

Explicit falsifiers (cross-experiment).

F1–F3 (E1), *F4–F5* (E2), *F6–F8* (E3) plus:

F9: λ coupling to SSI/CRP/SSE does **not** lower incident hazard vs. fixed λ (HR ≥ 0.95, p ≥ 0.05).

F10: Micro-MPC triggers show **no** improvement in J realized vs. baseline under trust-region parity.

F11: Early-warning bands (BH-1/BH-2) fail to improve **lead-time-adjusted** precision/recall vs. naive thresholds.

Reproducibility.

Pre-register knobs: ramp schedule, windows W_s / W_m / W_l, thresholds $\tau_{\text{sse}}, \tau_{\text{crp}}, \tau_{\Xi}$, controller budgets, and rollback rules.

Log to append-only store: per-tick L, Γ, λ, KL/Δlogit, chosen action, candidate table, health snapshot.

Publish replay notebooks and anonymized metrics for independent reruns.

Ethics & safety guardrails.

Canary exposure caps; auto-rollback on emergency band; tenant-aware throttles; human review escalation thresholds; privacy-preserving telemetry.

Outcome. If E1 shows breakpoints and hysteresis, E2 shifts thresholds right under buffering, and E3 reduces chattering via routing costs—*while falsifiers do not trigger*—we gain empirical support for surplus-aware control. If falsifiers hold, the framework is rejected or must be revised (e.g., decouple λ from SSI/CRP/SSE, retune micro-MPC, or redesign indicators).

8. Deployment Guide (Add-On Controller, Not a Model Rewrite)

Goal: bolt the **L–Γ controller** onto an existing inference stack **without retraining** or changing base weights. You'll insert a middleware between *logits* and *sampling*, add **minute-level telemetry** + **caps**, then roll out via **Shadow → Canary → Gradual** with unit tests and a crisp runbook.

8.1 Middleware insertion between logits and sampling

Where it lives. Right after you obtain baseline logits ℓ_t (or probs p_t) and **before** you sample the next token / choose a tool / emit a plan edge.

Responsibilities.

Compute per-tick features (from §3–§4): $\theta, \phi, \ell, iT, E_s$, logit entropy, mode transitions, latency.

Score candidates with $J_t(u) = L_t(u) - \lambda_t \, \Gamma_t(u)$.

Enforce a **trust region** vs. baseline (KL or ‖Δlogits‖), and run **micro-MPC** when triggered.

Emit the chosen action and a compact **audit packet**.

Minimal interface.

```
class DissipativeController:
    def __init__(self, cfg, policy, telemetry):
        self.cfg, self.policy, self.telemetry = cfg, policy, telemetry

    def step(self, logits, ctx, features):
        """
        logits: np.array[V] — baseline model logits at tick t
        ctx:    struct       — request metadata, tenant, caps, budgets
        features:            — {embeddings, goal g, projection basis B, SSE/CRP/
SSI window stats}
        returns: (action, audit)
        """
        # 1) Compute value & dissipation components
        L_parts = compute_L_parts(logits, ctx, features)       # likelihood,
progress, struct risk, etc.
        Gamma_parts = compute_Gamma_parts(logits, ctx, features) # drift, mode
flip, fmt debt, iT

        # 2) Adaptive λ and trust-region budgets (from minute-level telemetry)
        lam, kl_budget, dlogits_budget = self.telemetry.current_budgets(ctx)

        # 3) Candidate set + short lookahead if needed
        C = topK_candidates(logits, k=self.cfg.topk) | tool_candidates(ctx)
        if trigger_micro_mpc(features):
            C = micro_mpc_rollouts(C, horizon=self.cfg.h, features=features,
budgets=(kl_budget, dlogits_budget))

        # 4) Score and select under trust region
        scores = {u: L(u,L_parts) - lam * Gamma(u,Gamma_parts) for u in C}
        action = argmax_trust_region(scores, logits, kl_budget, dlogits_budget)

        # 5) Emit audit packet
        audit = {
            "L":L_parts, "Gamma":Gamma_parts, "lambda":lam,
            "kl_used": kl_div(action, logits), "dlogits_l2": dlogits(action,
logits),
            "candidates": len(C)
        }
        return action, audit
```

Trust region enforcement.

KL mode: project adjusted distribution q_t onto $\{\mathrm{KL}(q_t \,\|\, p_t) \le \varepsilon_t\}$.

Δlogits mode: clip the logit delta to $\|\Delta \ell_t\|_2 \le \delta_t$.

Pick one per deployment for simplicity; both are supported.

Streaming + tools.

Tick on punctuation/AST edges for text; tick at *call* and *return* for tools.

Add candidate "tool-call" actions with priors; include **routing costs** in Γ.

Failure-safe.

If features are missing or budgets exhausted: **fall back** to baseline sampling.

If NaNs or parser errors: **emit schema tokens** (headers/JSON braces) and stop-sequence.

8.2 Minute-level telemetry, caps, and safe defaults

Ingest every tick: `(tenant, t, L_parts, Γ_parts, λ, KL, Δlogits, θ, ϕ, iT, SSE, CRP, SSI, lat_ms, tool_ms, tokens)`.
Aggregate every minute (EMA smoothing) to drive budgets and alerts.

Telemetry service (contract).

```
{
  "minute_bucket": "2025-09-07T12:34:00Z",
  "SSE": 0.52, "CRP": 0.31, "SSI": 0.48, "Es_spike": false,
  "state": "Caution",
  "lambda": 0.64, "kl_budget": 0.010, "dlogits_budget": 0.9,
  "bh1": 0, "bh2": 0,
  "caps": {"qps": 30, "tok_max": 15000, "tool_per_min": 40}
}
```

Safe defaults (start here, tune nightly).

Controller: `topk=8`, `horizon H=5`, `γ=0.9`, `micro_mpc_trigger_on = Es_spike ∨ BH1 ∨ schema_boundary`.

λ schedule: $\lambda_0 = 0.4$, $\lambda_{\min} = 0$, $\lambda_{\max} = 2.0$, $a = b = 0.8$, $c = 0.5$. EMA smoothing 0.7.

Trust region: `KL_budget=0.01` *or* `Δlogits_l2=1.0` in Green; scale down 30–60% in Action.

SSE floor: `min_SSE=0.5` (below this, enforce exploration quota 10–20%).

Mode-flip shield: no more than 1 tool flip / 3 ticks under Action.

Fmt debt gate: if `FmtDebt>0.6` for 2 ticks → inject schema tokens; prefer shorter branches.

Caps by reservoir (auto-adapt with SSI/CRP).

M (material): tokens/sec, concurrent jobs, tool_ms/min.

F (financial): spend/min, spend/day; surge floors as optional.

I (institutional): reviewer-hours, escalations/h, policy-check QPS.

A (attention): notifications/min, active sessions, context reuse.

C (cognition): retries per task, allowed branches, edit passes.

Privacy & retention.

Hash logits and embeddings; store **metrics only** (no raw content) by default.

PII redaction on; per-tenant retention (e.g., 7–30 days) configurable.

Audit packets whitelisted fields only.

Dashboards (minute granularity).

Top: SSE/CRP/SSI with bands + BH-1/BH-2 markers.

Middle: λ, KL/Δlogits budgets used (%), mode-flip rate, FmtDebt.

Bottom: incidents, schema errors, latency p95/p99, throughput, caps hit rate.

8.3 Shadow → Canary → Gradual rollout; unit tests and runbook

A. Rollout ladder

Stage 0 — Replay Calibration (offline)

Fit α, β weights so median $L \approx \Gamma$ on healthy runs.

Backtest alert bands for **lead time** (target: ≥3–8 min interactive; ≥10–30 min batch).

Stage 1 — Shadow (0% user impact)

Run controller in parallel; **do not** affect sampling.

Compare realized baseline vs. counterfactual J, incidents, and early-warning quality.

Exit criteria:

≥10k sessions shadowed; ≥5% improvement in predicted J under matched KL; no uptick in CRP/SSI; alert lead time meets target.

Stage 2 — Canary (1–5% traffic, tenant-scoped)

Enable controller for a small slice; keep strict caps and fast rollback (R0–R3).

Run **A/B** with matching traffic; monitor per-minute.

Kill switches:

BH-2 sustained W_m; incident rate +50% vs. control; p95 latency +25%; schema break > 0.5% absolute.

Stage 3 — Gradual (5%→25%→50%→100%)

Expand only if the **previous tier** stabilizes for 24–48h and no emergency rollbacks occurred.

Widen KL budget slightly when stable; maintain exploration quotas until SSE baseline rises.

B. Unit tests (must pass before Canary)

Functional

TR-Enforce: rejects actions exceeding KL/Δlogits budgets.

L/Γ Monotonicity: increasing drift/mode flips raises Γ; better alignment raises L.

Schema Guards: malformed JSON/code triggers structure tokens within 2 ticks.

Micro-MPC Trigger: fires on E_s spikes and schema boundaries.

Stability

No-Feature Fallback: missing features → baseline sampling (0 failures in 10k ticks).

NaN/Inf Immunity: safe clamp + fallback path.

Burst Load: controller throughput ≥ X tokens/s with `topk=8, H=5`.

Safety

Caps Respect: simulated overload → caps lower budgets, reduce branching, throttle QPS.

Privacy: audit packet contains only whitelisted fields; PII redaction verified.

Determinism (optional)

With fixed RNG seeds & fixed budgets, identical audit packets for identical inputs.

C. Runbook (one-page, laminated)

Green → Caution (auto-nudge)

Telemetry raises λ 15–30%, tightens KL 20%.

L2 enforces exploration quota if SSE<0.5; freezes tool chattering.

Owner checks dashboards; no user visible change expected.

Caution → Action (BH-1)

Enable micro-MPC; route away from hot shards; apply soft caps.

Start buffer rebuild: allocate reviewer minutes, add rest windows.

Log *potential incident*; begin 30-min observation.

Action → Emergency (BH-2)

Execute **R2→R5** (from §5): throttle/route, degrade gracefully, increase HITL, optionally suspend tenant/feature.

Snapshot config; open incident ticket with last 60 min of audit packets.

When SSE>0.55 & CRP below threshold for 30 min, begin staged recovery.

Post-incident

Blameless review; retune bands/weights; add synthetic tests reproducing pattern.

Update policy packs, exploration quotas, and caps if concentration caused the event.

D. Config templates

Controller (YAML)

```yaml
controller:
  topk: 8
  horizon: 5
  gamma: 0.9
  micro_mpc_triggers: ["Es_spike", "BH1", "schema_boundary"]
  lambda:
    base: 0.4
    min: 0.0
    max: 2.0
    weights: {SSI: 0.8, CRP: 0.8, invSSE: 0.5}
    ema: 0.7
  trust_region:
    mode: "kl"          # or "dlogits"
    kl_budget_green: 0.010
    kl_budget_action: 0.006
    dlogits_green: 1.0
    dlogits_action: 0.7
  exploration:
    min_SSE: 0.50
    quota_pct: 0.15
  mode_flip_guard:
    min_ticks_between_flips: 3
```

Policy & caps (JSON)

```
{
  "caps": {"qps": 30, "tok_max": 15000, "tool_per_min": 40},
  "routing": {"min_spread": 0.55, "providers": ["P1","P2"]},
  "buffers": {"reviewer_hours": 12, "rest_windows": "nightly"},
  "privacy": {"pii_redact": true, "retention_days": 14}
}
```

Outcome. With this add-on controller, you deploy **surplus-aware decoding** as a thin layer: a stable trust-region wrapper, minute-level health adaptation, and clear rollout/rollback discipline. You get measurable improvements in quality and stability without touching model weights—and a paper trail that auditors (and on-call engineers) can actually use.

9. Governance & Ethics

Goal: keep value creation high **without** burning residual capacity, eroding consent, or letting ecosystems collapse into concentration traps. Governance is part of the same closed loop: **metrics → control → disclosure → incentives**.

9.1 Data compensation & buffer funds during high-risk phases

A. Who gets compensated

Contributors: dataset owners, annotators/reviewers, tenants whose interactions are reused, and federated partners.

Attribution: every training/replay/improvement event must carry a **Contribution ID** (hashable provenance bundle) and a **Consent flag** (active, scoped, revocable).

B. How to pay (auditable formula)

$$\text{Payout}_i = r_{\text{base}}\Big[u_i + w_q\,\Delta\,Q_i + w_n\,\text{Novelty}_i\Big]\cdot s_i$$

u_i: usage weight (queries, tokens, coverage).

$\Delta\,Q_i$: measurable quality uplift attributable to i (offline replay or online A/B).

Novelty_i: de-dup / diversity score (reduces overpayment for near duplicates).

s_i: safety scalar (downweights content that drives rework/violations).

Ledger: monthly export, per tenant/provider, with confidence bands.

C. Dynamic Buffer Fund (for review time, escalation, rest windows)

$$\text{Reserve}_t \;=\; \rho_t \cdot \text{RollingSpend}_t, \qquad \rho_t = \rho_0 + a\,\widetilde{\text{CRP}}_t + b\,(1-\text{SSE}_t) + c\,\widetilde{\text{SSI}}_t^{\star}$$

Triggers: **BH-1** → +25% ρ_t for 1h; **BH-2** → +50% and immediate fund unlock for HITL and cooldown.

Ring-fencing: Buffer funds are separate from feature budgets; unspent balance rolls forward.

D. Consent & deletion guarantees

Revocation SLA: content fingerprinted; removal cascades through caches, replays, and federated shards within T_{del} (e.g., 30 days).

Negative balance shield: payout clawbacks never exceed the month's accruals; disputes go to an independent review pool.

9.2 Transparent audit (KL/Δlogit, triggers, bias gaps) & incident disclosure

A. "Controller Card" (per release)

Objective weights (α_i, β_i), λ schedule, trust-region budgets, exploration quota policy, routing costs.

Validation: backtests for **lead time**, false-positive rate, and impact on $J = L - \lambda\,\Gamma$.

B. Per-tick audit packet (stored, redacted)

L and Γ breakdowns; λ_t; KL/Δlogit used vs. budget; trigger flags (BH-1/BH-2, E_s spike); chosen action; caps hit.

Privacy: metrics only by default (hashed embeddings/logits), PII redaction, tenant-scoped retention.

C. Bias gap monitoring

Gap index (BGI) per protected attribute or content class:

$$\text{BGI} = \left|\text{PassRate}_A - \text{PassRate}_B\right| \quad \text{with CI and drift alarms}$$

If BGI exceeds τ_{bgi}: raise λ on dissipation that correlates with gap drivers (e.g., mode flips, repair debt), add targeted evaluation sets, and publish remediation notes.

D. Incident disclosure (for BH-2 or material harm)

What to publish: timeline, precursors (SSE/CRP/SSI, Ξ), decisions (changes to λ, KL budgets, caps), user impact, root cause, corrective actions, new thresholds, and rollback path used (R0–R6).

When: within 7 days for external notices; internal post-mortem in 72 hours, blameless format.

E. Access & segregation

Read-only audit store; dual-control on export; federated shards keep local keys.

On-call runway: last 60 minutes of packets retrievable in ≤ 30 seconds.

9.3 Anti-concentration routing & federated exchange principles

A. Anti-concentration policy (avoid semantic black holes)

Minimum spread constraints: enforce $\mathrm{SSE} \geq \tau_{\mathrm{sse}}$ per tenant/feature and globally; if breached, route to diverse providers/plans until recovered.

Provider and path quotas: no single provider or toolchain > q max (e.g., 40%) for a class of tasks in Action/Emergency states.

Circuit breakers: if a shard's $\mathrm{SSE} < 0.4$ **and** $\mathrm{CRP} > \tau_{\mathrm{crp}}$ for W_m, freeze new intake and drain to alternates.

B. "Semantic currency" for handoffs (priced coordination)

$$\Gamma_t \leftarrow \Gamma_t + \sum_{\text{handoffs } i \to j} \rho_{i \to j}, \quad \rho_{i \to j} = \rho_0 \cdot g\left(\widetilde{\mathrm{SSI}}, \widetilde{\mathrm{CRP}}\right)$$

Makes excessive cross-agent hopping expensive under stress, reducing chattering while leaving healthy exploration intact.

C. Federated exchange (decentralize power, centralize accountability)

Data stays local; models exchange **gradients/metrics** with secure aggregation; contribution credits flow via the ledger.

Portability tokens: contributors can move credits across providers; exchange fees top up the Buffer Fund.

Neutral schemas: shared definitions for audit packets, consent flags, and payout fields; no opaque ranking APIs.

No single chokepoint: multi-provider routing by design; failover tested quarterly.

D. Governance cadence

Quarterly **Safety & Equity Review**: BGI trends, concentration metrics, payout fairness, incident catalog.

External advisory panel for contested decisions (payout disputes, sensitive routing policies).

Outcome. Governance is not an appendix; it is the fourth leg of the loop. By tying **compensation** to measurable uplift and diversity, funding **buffers** when risk rises, **auditing** every control decision, and preventing **concentration**, the system earns trust while preserving the residual capacity it needs to keep creating value.

10. Limitations, Risks, and Mitigations

This section is intentionally pragmatic: where the framework can fail, how to notice early, and what to do. Each item pairs a **failure mode** with **mitigations you can ship**.

10.1 Scope of near-linearity, proxy errors, clustering sensitivity

A) "Near-linearity" is local and brittle

Risk. The "semantic black-hole" interior appears locally linear, but only within a small neighborhood. Extrapolating linear behavior across sections, tools, or user cohorts can mis-steer the controller.

Mitigations

Linearity gate. Fit a local linear model for $\Delta\theta$ on the last W_s ticks; require $R^2 \geq 0.85$ *and* low SSE before using linear approximations (otherwise fall back to generic micro-MPC).

Dwell & deadbands. Don't toggle modes unless the linearity gate holds for $\geq k$ consecutive ticks and relax only after SSE $> \tau_{\text{sse}} + \delta$.

Diversity bump on exit. When leaving a near-linear regime, temporarily raise exploration quota (+5–10%) to rebuild spread and avoid relock.

B) Proxy error in L and Γ

Risk. L (value) and Γ (dissipation) are proxies. Mis-specified weights or stale predictors can reward "pretty but useless" output or over-penalize productive tool switches.

Mitigations

Monthly re-grounding. Recalibrate α_i, β_i against **human-rated** samples and task success labels (held-out tenants).

Split immediate vs. future costs. Keep structure risk in L (now) and repair debt in Γ (later) to avoid double counting.

Ablation audits. For a random 5% of traffic, log "counterfactual J" under $\pm 20\%$ weight perturbations; alert if sign flips occur >2% of the time (proxy instability).

Pareto check. If a change improves L while worsening Γ (or vice versa), require micro-MPC to demonstrate net J gain under the trust region before adoption.

C) Clustering sensitivity in SSE

Risk. SSE depends on clustering choices (N, initialization, drift). Poor settings can mis-detect concentration.

Mitigations

Consensus SSE. Compute SSE over an **ensemble** of 3–5 clusterings (different seeds / random projections) and average.

Adaptive N. Set cluster count by silhouette/DBCV peaks per tenant; re-evaluate weekly.

k-NN entropy fallback. When clusters are unstable, estimate spread with k-NN entropy on embeddings (no clustering).

Concept drift guard. If topic centroids move $>\delta$ in a week, rotate bases and re-seed clusters; keep a two-week overlap window to avoid step changes.

10.2 Oscillation risks from tight coupling; Goodhart pressures

A) Control oscillations (λ, KL budgets)

Risk. Tight coupling $\lambda = f(\text{SSI}, \text{CRP}, \text{SSE})$ with delays can cause **hunting**: the controller over-tightens, traffic re-routes, indicators swing back, and the loop oscillates.

Mitigations

Low-pass + slew limits. EMA smooth λ ($\beta \approx 0.7–0.85$) and cap $|\lambda_t - \lambda_{t-1}| \leq \Delta_{\max}$.

Hysteresis bands. Different enter/exit thresholds for BH-1/BH-2; require minimum **dwell time** before changing state.

PI, not D. Use proportional–integral updates to λ; avoid derivative terms on noisy indicators.

Budget ramping. Adjust KL/Δlogit budgets in ≤10–20% steps per minute; never both widen $*and*$ lower λ in the same minute.

B) Goodhart and gaming

Risk. If teams are measured on a single headline metric (e.g., SSE or incident count), they may optimize that proxy at the expense of others (e.g., suppress complex tasks to keep incidents low).

Mitigations

Rotating indicator sets. Every quarter, rotate one metric in each family (spread, fragility, saturation) while keeping invariants (privacy, schema validity).

Composite objectives. SLOs are **AND-gated**: quality, latency, and concentration must all meet targets for a "win."

Randomized audits. With small probability p, override controller choices to collect counterfactuals for drift detection; penalize policies that degrade hidden hold-outs.

Incentive alignment. Tie data compensation and buffer funding (Section 9) to **uplift across metrics**, not a single KPI.

10.3 Multi-objective tuning, rate-limiters, rotating indicators, graded rollback

A) Multi-objective tuning

Risk. Weighted sums hide trade-offs; the same α, β may not fit all tenants or tasks.

Mitigations

Tenant-specific priors. Learn α, β per tenant/feature with hierarchical shrinkage; audit that global safety constraints still hold.

Lexicographic constraints. Enforce **hard** schema/privacy constraints first, then optimize J within the feasible set.

Pareto front review. For monthly tuning, present Pareto curves (quality vs. cost vs. dissipation) and pick operating points with explicit governance sign-off.

B) Rate-limiters & back-pressure (operational)

Risk. Even correct control decisions fail if the plant saturates.

Mitigations

Slew-rate on action complexity. Limit increases in branching width, tool concurrency, and context length per minute.

Back-pressure path. When $\mathrm{SSI}^\star$ breaches, propagate "slow down" upstream: smaller prompts, shorter horizons, defer non-critical tool calls.

Graceful degrade ladders. Pre-define templates/short answers; flip to them under Emergency without violating schemas.

Reference snippet

```
if state == "Action":
    kl_budget *= 0.7
    branch_width = max(1, int(branch_width*0.7))
    tool_concurrency = max(1, tool_concurrency - 1)
if state == "Emergency":
    use_templates = True
    exploration_quota = min(exploration_quota, 0.05)
```

C) Rotating indicators (resilience to drift)

Risk. Static indicator definitions get stale.

Mitigations

Quarterly rotation. Swap in alternative fragility proxies (e.g., tail latency variance, reviewer disagreement rate) and spread proxies (k-NN entropy vs. clustered SSE).

Shadow metrics. Track 1–2 extra indicators without using them for control; promote only after stability tests.

D) Graded rollback (avoid cliff-edge behavior)

Risk. All-or-nothing rollbacks either act too late or cause unnecessary downtime.

Mitigations

R0→R6 ladder (Section 5) with **explicit exit criteria** per rung (e.g., SSE > 0.55 for 30 min and CRP below threshold).

Freeze hazardous axes first. On BH-2: freeze tool flipping and horizon length before cutting QPS.

Partial tenant suspension. Throttle hot routes/skills within a tenant before full tenant pause.

Quick checklist (ship with the controller)

Linearity gate + dwell/exit rules implemented.

Weight re-grounding job (monthly) + ablation alarms on proxy instability.

Consensus SSE + k-NN entropy fallback wired.

λ smoothing + slew limits + hysteresis on BH-1/2; budget ramp steps capped.

Rotating indicators schedule and shadow metrics in place.

Rate-limiters for branching, tools, context; back-pressure hooks implemented.

Graded rollback with runbook and exit criteria tested in drills.

Bottom line. Treat indicators as **instruments, not truths**; keep the loop slow enough to be stable, fast enough to be useful; and practice rollbacks until they are boring.

11. Roadmap

This roadmap gets a surplus-aware controller from lab to production without touching base model weights. It's split into **Alpha (replay + shadow)**, **Beta (canary + pilots)**, and **GA (productization)**, with crisp artifacts and exit criteria. A second part standardizes **open instrumentation** so multiple teams/providers can interoperate safely.

11.1 Alpha (replay + shadow), Beta (canary + pilots), GA (productization)

Alpha — Replay & Shadow (Weeks 0–4)

Objective: prove the loop works without user impact.

Build

Adapters: extract per-tick features (θ, φ, iT, SSE/CRP/SSI, logit entropy), audit packet, and health snapshot.

Weights: calibrate α, β for L and Γ; set λ schedule; choose **one** trust-region mode (KL *or* Δlogits).

Backtests: run E1–E3 on historical traces; estimate lead-time curves for BH-1/BH-2.

Controller Card v0.1: document L & Γ terms, λ function, budgets, and caps.

Shadow run

Run the controller in parallel (no influence on sampling).

Log **counterfactual** J and trust-region usage against baseline.

Exit criteria

≥10k shadowed sessions across ≥3 tenants.

+5–10% median counterfactual J at matched KL/Δlogits.

Lead time: BH-1 median ≥3–8 min (interactive) or ≥10–30 min (batch).

No increase in proxy incident rates on matched replay.

Beta — Canary & Pilots (Weeks 5–10)

Objective: demonstrate live benefit at small scale with rollback discipline.

Canary (1–5% traffic, tenant-scoped)

Enforce **R0–R3** rollback ladder; enable **micro-MPC** on BH-1; apply exploration quotas when SSE low.

Activate **Buffer Fund** for on-call reviewer minutes during Action/Emergency.

Pilots

2–3 scoped workloads (e.g., code assistant, customer support, data extraction).

Run **E2/E3** interventions on a slice to validate buffer and routing-cost effects.

Kill switches

Sustained BH-2 over W_m.

+50% incident rate vs. control, or p95 latency +25%.

Schema validity drop >0.5% absolute.

Exit criteria

Hazard ratio to incident (BH-2→incident) ≤ 0.8 vs. control ($p<0.05$).

Mode-flip rate ↓ and **FmtDebt** ↓ at matched quality.

SSE baseline ≥ +0.05 in pilots without CRP/SSI worsening.

Successful **R0–R3** drills; no Emergency rollback required for ≥7 days.

GA — Productization (Weeks 11–16)

Objective: make it boring: multi-tenant, auditable, maintainable.

Deliver

Multi-tenant tuning: hierarchical α, β with global guardrails; per-tenant caps & routing.

Ops: on-call rotations, SLOs (quality/latency/concentration), quarterly **Safety & Equity Review**.

Governance: payout ledger live, incident disclosure workflow, consent & deletion SLA wired.

Hardening: deterministic test seeds, throughput benchmarks, privacy redaction, tamper-evident logs.

Exit criteria

≥50% traffic on controller for 14 days, then gradual to 100% with no GA rollbacks.

SLOs met across quality/latency/concentration; audit exports reproducible in <30s.

External "Controller Card" published and signed off by governance.

11.2 Open Instrumentation Standards and Reference Configs

To interoperate across teams/providers, we standardize **events, fields, and configs**. The defaults avoid raw content; they're privacy-preserving but rich enough for audit and control.

A) Event Schemas (SemVer: `smfc-telemetry/1.x`)

A1. Controller Audit Packet (per tick) — `cap.v1`


```
{
  "ts":"2025-09-07T12:34:56Z","tenant":"acme","session":"s-abc","model":"gpt-x",
  "hash_logits":"h:p_t","hash_embed":"h:e_t",
  "theta":0.29,"phi":0.83,"iT":0.21,"Es":0.17,
  "L":{"dlp":0.34,"progress":0.18,"structRisk":0.03,"lat":120,"len":18},
  "Gamma":{"drift":0.05,"modeFlip":0.00,"fmtDebt":0.02,"iT":0.21},
  "lambda":0.72,"kl_used":0.012,"dlogits_l2":0.90,
  "action":{"type":"token|tool|plan","name":"…","args_hash":"h:…"},
  "candidates":6,"caps_hit":false,"state":"Caution","bh1":0,"bh2":0
}
```


A2. Health Snapshot (per minute) — `health.v1`


```
{
  "bucket":"2025-09-07T12:34:00Z","tenant":"acme",
  "SSE":0.52,"CRPz":0.31,"SSIstarZ":0.28,"Xi":0.41,
```

```
  "lambda":0.64,"kl_budget":0.010,"dlogits_budget":0.90,
  "state":"Caution","bh1":0,"bh2":0
}
```

A3. Policy Pack (caps & routing) — `policy.v1`

```
{
  "caps":{"qps":30,"tok_max":15000,"tool_per_min":40},
  "routing":{"min_spread":0.55,"providers":["P1","P2"]},
  "exploration":{"min_SSE":0.50,"quota_pct":0.15},
  "privacy":{"pii_redact":true,"retention_days":14}
}
```

A4. Consent & Credit Ledger — `credit.v1`

```
{
  "contrib_id":"cid:…","consent":"active|scoped|revoked",
  "usage":{"queries":123,"tokens":45678,"coverage":0.31},
  "uplift":{"delta_quality":0.07,"ci":[0.03,0.10]},
  "novelty":0.62,"safety_scalar":0.93,"payout":12.34,"currency":"USD"
}
```

Principles

No raw text by default; hashes for logits/embeddings.

Tenant partitioning by design; export is **dual-control**.

Tamper-evidence: hash-chain over CAPs; health snapshots sign the last CAP hash.

Clocking: all timestamps ISO-8601 Zulu; buckets are closed intervals.

B) Reference Configs (drop-in starters)

B1. Controller (YAML) — `controller.v1.yaml`

```
controller:
  topk: 8
  horizon: 5
  gamma: 0.9
  micro_mpc_triggers: ["Es_spike","BH1","schema_boundary"]
  lambda:
    base: 0.4
    min: 0.0
    max: 2.0
    weights: {SSI: 0.8, CRP: 0.8, invSSE: 0.5}
```

```
    ema: 0.7
  trust_region:
    mode: "kl"            # or "dlogits"
    kl_budget_green: 0.010
    kl_budget_action: 0.006
    dlogits_green: 1.0
    dlogits_action: 0.7
  exploration:
    min_SSE: 0.50
    quota_pct: 0.15
  mode_flip_guard:
    min_ticks_between_flips: 3
```

B2. Telemetry Aggregator — `telemetry.v1.yaml`

```
telemetry:
  window_short_s: 120
  window_med_min: 10
  window_long_h: 4
  smoothing_ema: 0.75
  debounce_ticks: 3
  thresholds:
    sse: 0.45
    crp_z: 0.8
    xi: 0.65
  exports:
    cap_stream: "kafka://cap.v1"
    health_stream: "kafka://health.v1"
    retention_days: 14
```

B3. Incident Thresholds — `incidents.v1.yaml`

```
incidents:
  bh1_auto: ["raise_lambda","tighten_kl","enable_mpc","diversify"]
  bh2_auto:
["route_throttle","degrade_tools","hitl_on","suspend_tenant_optional"]
  exit_criteria:
    recover_sse: {min: 0.55, duration_min: 30}
    recover_crp: {max_z: 0.3, duration_min: 30}
```

C) Interop Tests (must pass before cross-provider routing)

Schema conformance (`cap.v1`, `health.v1`, `policy.v1`, `credit.v1`).

Budget parity: KL/Δlogits measurement matches within ±5% on a common test set.

Alert parity: BH-1/BH-2 fire identically on a shared replay bundle.

Privacy: redaction & retention honored; synthetic PII never leaves the tenant shard.

Determinism: fixed seeds → byte-identical CAPs on a golden run.

D) Publishing & Governance

Controller Card (per release): public summary of objectives, budgets, alert bands, and known limitations.

Open notebooks: replay calibration and survival analysis (anonymized metrics).

Quarterly rotation: alternative fragility/spread proxies trialed as **shadow metrics**; promoted only after stability review.

Change control: any change to λ function, alert thresholds, or caps is versioned and tied to an incident drill.

Outcome. With an evidence-based rollout (Alpha → Beta → GA) and a small set of open, privacy-preserving schemas and configs, teams can ship **surplus-aware control** as an add-on, interoperate across providers, and keep the loop auditable end-to-end.

Appendix A — Symbols & Notation (variables, indices, units)

Conventions. Scalars *italic*, vectors **bold**, operators ^hatted^. All angles in radians unless noted. Logs are natural logs. Time advances in **semantic ticks** (τ); minute-level buckets are used for telemetry. Ranges below are indicative defaults.

A1. Indices, time, and sets

Symbol	Type / Units	Meaning / Definition	Notes
t	integer tick	Semantic tick index	1 tick ≈ 10–40 tokens **or** one tool decision
τ	scalar	Semantic tick-time unit	Used for finite differences (Δ/Δτ)
W_s, W_m, W_l	time windows	Short / Medium / Long rolling windows	E.g., 30–180 s; 5–15 min; 1–6 h
k	index	Flow type $k \in \{M, F, I, A, C\}$	Material, Financial, Institutional, Attention, Cognition
$\mathbb{1}\{\cdot\}$	indicator	1 if condition true else 0	Used in SSI
N	count	#clusters for semantic spread	10–200 typical

A2. Reservoir dynamics (surplus flows)

Symbol	Type / Units	Meaning / Definition	Notes
$R_k(t)$	scalar	**Residual** stock for flow k	Capacity "room to maneuver"
$G_k(t)$	scalar	Generation/replenishment into R_k	From value-creating actions
$L_k(t)$	scalar	Loss/consumption from R_k	From tokens, latency, reviews…
μ_k	1/tick	Background leakage rate	Fatigue, contention, decay
$C_{k\to j}(t)$	scalar	Cross-type conversion spend	Exchange rate $\alpha_{k\to j}$ applies
$K_k(t)$	scalar	Effective capacity cap for k	Hard/soft; queue effects
$x_k(t)$	scalar	Activity proxy for k	Requests/ min, tool calls/min
$Q_k(t)$	scalar	Queue/backlog for k	Items or minutes outstanding
Update	—	$R_k(t{+}1) = \min\{R_k + G_k - L_k - \mu_k R_k + \sum_{j\neq k}(C_{j\to k} - C_{k\to j}),\ K_k\}$	Capacity-limited residual update

A3. Semantic geometry (state & kinematics)

Symbol	Type / Units	Meaning / Definition	Notes
$\mathbf{e}_t \in \mathbb{R}^d$	vector	Rolling embedding of current content	Sentence/phrase or tool context
$\mathbf{g} \in \mathbb{R}^d$	vector	Goal vector from brief/ instructions	Average of section headers, schema names
θ_t	angle	Orientation misalignment	$\cos\theta_t = \langle \mathbf{e}_t, \mathbf{g}\rangle / (\|\mathbf{e}_t\|\|\mathbf{g}\|)$

Symbol	Type / Units	Meaning / Definition	Notes
$\hat{\mathbf{0}}$	operator	Projection onto constraint subspace S	Built from basis $B = \{\mathbf{s}_i\}$
ϕ_t	scalar [0, 1]	In-subspace proportion	$\phi_t = \|\hat{\mathbf{0}}(\mathbf{e}_t)\|/\|\mathbf{e}_t\|$
ℓ_t	scalar [0, 1]	Orthogonal leakage	$\ell_t = \sqrt{1-\phi_t^2}$
iT_t	scalar $[0, \infty)$	**Semantic tension** (stored misalignment/concentration)	$iT = \alpha(1-\cos\theta) + \beta\,\ell + \gamma(1-\text{SSE})$
$U(\theta)$	scalar	Goal-mismatch potential	$U = \frac{1}{2}\kappa(1-\cos\theta)^2$
κ	scalar	Curvature (restoring strength)	Fitted on replay
ω_t	angle/tick	Angular rate	$\omega \approx \Delta\theta/\Delta\tau$
m_s	scalar	**Semantic mass** (inertia)	$m_s \approx I/(\Delta^2\theta/\Delta\tau^2)$
F_s	scalar	**Semantic force**	$F_s = -\partial U/\partial\theta \approx -\kappa\theta$ (small θ)
T_s	scalar	"Kinetic" term	$T_s = \frac{1}{2}m_s\omega^2$
E_s	scalar	**Semantic energy**	$E_s = T_s + U$ (interrupt gate)

A4. Decoding, distributions, and entropy

Symbol	Type / Units	Meaning / Definition	Notes
ℓ_t	vector	Baseline logits at tick t	From base model
p_t	dist.	Baseline token distribution	Softmax(ℓ_t)
q_t	dist.	Adjusted distribution under control	Trust-region bounded
$\mathrm{KL}(q \,\|\, p)$	scalar	KL-divergence constraint	Budget ε_t
Δlogits_t	vector	Logit delta norm	Budget δ_t (L2)
H_t	scalar	Token entropy	$H = -\sum_v p_t(v)\log p_t(v)$
C_t^{logit}	scalar	Logit concentration	(1 - H_t/\log

A5. Objective & penalties (per tick)

Symbol	Type / Units	Meaning / Definition	Notes
$J_t(u)$	scalar	Per-candidate score	$J = L - \lambda\Gamma$
$L_t(u)$	scalar	**Value** term	Likelihood gain + progress – structure/latency/length

Symbol	Type / Units	Meaning / Definition	Notes
$\Gamma_t(u)$	scalar	**Dissipation** term	Drift + mode flip + format debt + iT carryover
λ_t	scalar $[0, \lambda_{\max}]$	Trade-off weight	$\lambda = f(\text{SSI}, \text{CRP}, \text{SSE})$
α_i, β_i	weights	Component weights for L, Γ	Tuned on replay
$\Delta \log p$	scalar	Likelihood gain	Relative to local baseline
Prog_t	scalar	Progress toward goal/ constraints	$\propto \Delta \cos\theta + \Delta\phi$
StructRisk	prob.	Immediate schema/ format break risk	Classifier score
Latency, Length	ms, tokens	Incremental runtime & tokens	Normalized for L
Drift	scalar	Predicted ↑ in θ and/or ℓ	Lookahead
ModeFlip	count	Rapid tool/mode switching penalty	EMA transitions
FmtDebt	scalar	Accumulated format repair debt	Decays when repaired

A6. Health indicators & alerts

Symbol	Type / Units	Meaning / Definition	Notes
SSI	[0,1]	**Saturation–Stress Index**	Utilization, backlog, cap hits
CRP	norm.	**Collapse Readiness Proxy**	Backlog growth, rework, CV of turnaround, near-miss rate
SSE	[0,1]	**Semantic Spread Entropy**	$-\sum_i p_i \log p_i / \log N$
Ξ_t	scalar	Composite risk	$a(1-\text{SSE}) + b\,\widetilde{\text{CRP}} + c\,\widetilde{\text{SSI}}^{\star}$
$\tilde{\cdot}$	—	Normalized (z-score or min–max)	Tenant-scoped preferred
$\text{SSI}^{\star}$	scalar	Max/p95 SSI across shards	Conservative stress proxy
BH-1	alert	Early black-hole alert	$\text{SSE} \le \tau_{\text{sse}} \wedge \widetilde{\text{CRP}} \ge \tau_{\text{crp}}$ over W_s
BH-2	alert	Severe/composite alert	$\Xi \ge \tau_{\Xi}$ sustained over W_m
State	enum	Health band	{Green, Caution, Action, Emergency}

Typical thresholds (tune per deployment): $\tau_{\text{sse}} \approx 0.45$, $\tau_{\text{crp}} \approx \ +0.8\sigma$, $\tau_{\Xi} \approx 0.65$.

A7. Micro-MPC & horizon

Symbol	Type / Units	Meaning / Definition	Notes
H	steps	Lookahead horizon	3–10 ticks typical
γ	scalar	Short-horizon discount	0.8–0.95
ε_t	scalar	KL budget	Tighten under stress
δ_t	scalar	Δlogits L2 budget	Alternative to KL
Trigger	logic	When to run MPC	E_s spike, BH-1, schema boundary

A8. Routing, costs, and buffers

Symbol	Type / Units	Meaning / Definition	Notes
$\rho_{i \to j}$	scalar	Routing cost for handoff $i \to j$	Added to Γ; adaptive under stress
q max	share	Max provider/path share	Anti-concentration cap (e.g., 40%)
Reserve_t	currency	Buffer fund reserve	$\rho_t \cdot \text{RollingSpend}$
ρ_t	scalar	Reserve ratio	Increases with $\widetilde{\text{CRP}}$, $(1 - \text{SSE})$, $\widetilde{\text{SSI}}^{\star}$

A9. Statistics & evaluation

Symbol	Type / Units	Meaning / Definition	Notes
HR	scalar	Hazard ratio	BH-2→incident risk vs. control
$\text{CV}[T_{\text{turn}}]$	scalar	Coef. of variation of turnaround	In CRP
σ	scalar	Standard deviation	For z-scoring
p	prob.	Randomized audit sampling prob.	e.g., 1–5%
$\Delta^{+} x$	scalar	Positive difference	$\max(0, x_t - x_{t-1})$

A10. Units & scaling conventions

Angles θ, ω: radians; report degrees only in UIs.

Rates: per tick unless noted; minute buckets for telemetry.

Probabilities: $[0, 1]$; entropies normalized by $\log N$ or $\log |V|$.

Budgets: KL in **nats**; Δlogits as L2 norm in logit units.

Normalization: tilde variables $\tilde{x}$ are tenant-scoped z-scores unless stated.

Privacy: embeddings/logits are stored as **hashes**; audit packets carry metrics only.

Master equation references (for quick lookup)

Residual update: see **A2**.

Objective: $J = L - \lambda\,\Gamma$.

Composite risk: $\Xi = a(1 - \mathrm{SSE}) + b\,\widetilde{\mathrm{CRP}} + c\,\widetilde{\mathrm{SSI}}^{\star}$.

Energy gate: $E_s = \frac{1}{2} m_s \omega^2 + \frac{1}{2} \kappa (1 - \cos\theta)^2$.

Trust regions: $\mathrm{KL}(q \,\|\, p) \le \varepsilon$ **or** $\|\Delta\mathrm{logits}\|_2 \le \delta$.

This appendix is designed to make audits, experiments, and implementation **unambiguous** across teams and providers.

Appendix B — Reference Pseudocode (Γ-Lite, Triggers, Trust Region) and YAML Profiles

This appendix gives drop-in pseudocode you can adapt to most inference loops. It includes a **Γ-Lite** (minimal dissipation) scorer, **trigger logic** for early-warning, **trust-region** enforcement, an **optional micro-MPC** rollout, and **YAML profiles** to wire it all together. It is self-contained and does not require model retraining.

B.1 Γ-Lite: a minimal dissipation scorer

Purpose. Cheap, robust approximation of Γ using only signals you already have: alignment drift, tool/mode flips, and format debt. Works when embeddings and a simple structure checker are available.

```
# ===== Γ-Lite components =====

def drift_score(theta_t, theta_tp1, leak_t, leak_tp1, w_theta=1.0, w_leak=0.5):
    dtheta = max(0.0, theta_tp1 - theta_t)     # misalignment increase
    dleak  = max(0.0, leak_tp1  - leak_t)      # off-subspace leakage increase
    return w_theta * dtheta + w_leak * dleak

def modeflip_score(last_modes, min_ticks_between_flips=3):
    """
    last_modes: deque of recent mode/tool ids (e.g., ['GEN','TOOL_A','GEN',...])
    Penalize A→B→A thrash and flips occurring too soon.
    """
    flip = 0.0
    if len(last_modes) >= 3 and last_modes[-3] == last_modes[-1] !=
last_modes[-2]:
        flip += 1.0                                  # A→B→A chatter
```

```
    # Cooldown: discourage flips within a short dwell
    if recent_flip_within(last_modes, min_ticks_between_flips):
        flip += 0.5
    return flip

def fmt_debt_score(current_debt, just_repaired: bool):
    """
    current_debt: running meter in [0, 1], increases on borderline tokens,
                  decreases when we emit structure-repair tokens.
    """
    if just_repaired:
        return max(0.0, current_debt - 0.2)     # decay on repair
    return current_debt

def gamma_lite(theta_t, theta_tp1, leak_t, leak_tp1, last_modes, fmt_debt,
               weights=dict(drift=1.0, flip=0.7, debt=0.6), **kwargs):
    drift = drift_score(theta_t, theta_tp1, leak_t, leak_tp1, **kwargs)
    flip  = modeflip_score(last_modes)
    debt  = fmt_debt_score(fmt_debt, just_repaired=False)
    return (weights["drift"] * drift
          + weights["flip"]  * flip
          + weights["debt"]  * debt)
```

Notes.

`theta` = orientation angle to goal; `leak` = orthogonal leakage to constraint subspace (§3).

`fmt_debt` increments when tokens drift off schema; decrement when you emit guard tokens (e.g., closing } or a heading).

B.2 Per-tick controller step (L–Γ with Γ-Lite)

```
def controller_step(logits_base, features, budgets, cfg):
    """
    logits_base : np.array[V]   # baseline logits at tick t
    features    : dict          # {theta_t, leak_t, goal g, proj basis B, SSE,
CRP, SSI, last_modes, fmt_debt, ...}
    budgets     : dict          # {'kl': eps_t, 'dlogits': delta_t}
    cfg         : ControllerCfg # weights, topK, horizon, trust-region mode,
etc.
    returns: (action, audit_packet)
    """
```

```
    # 0) Precompute cheap features
    p_base   = softmax(logits_base)
    H        = -np.sum(p_base * np.log(p_base + 1e-12))        # entropy
    conc     = 1 - H/np.log(len(p_base))

    # 1) Compute λ from health (minute-level) or from provided features
    lam = compute_lambda(features["SSI"], features["CRP"], features["SSE"],
cfg.lambda_sched)

    # 2) Candidate set (topK tokens + tool calls)
    C_tok  = topk_indices(logits_base, k=cfg.topk)
    C_tool = tool_candidates(features, cfg)                     # optional
    C      = make_actions(C_tok, logits_base) | C_tool

    # 3) For each candidate, estimate L and Γ-Lite
    scored = []
    for u in C:
        theta_tp1, leak_tp1, struct_risk, lat_ms, length =
cheap_one_step_estimates(u, features)
        # Value (L): auditable sum
        L = ( cfg.alpha.dlp        * delta_logp(u, logits_base)    # likelihood
gain
            + cfg.alpha.progress   * progress(theta_tp1, features["theta_t"],
leak_tp1, features["leak_t"])
            - cfg.alpha.structRisk * struct_risk
            - cfg.alpha.latency    * norm_latency(lat_ms)
            - cfg.alpha.length     * norm_length(length) )

        # Dissipation (Γ-Lite)
        Gamma = gamma_lite(features["theta_t"], theta_tp1,
                           features["leak_t"],  leak_tp1,
                           features["last_modes"],
                           features["fmt_debt"],
                           weights=cfg.beta.to_dict())
        scored.append((u, L - lam * Gamma, L, Gamma))

    # 4) Trust-region selection vs. baseline
    action = argmax_under_trust_region(scored, logits_base, budgets,
cfg.trust_region)
```

```
    # 5) Build audit packet
    audit = {
        "L_parts": {...}, "Gamma_parts": {...}, "lambda": float(lam),
        "kl_used": kl_divergence_of(action, logits_base) if
cfg.trust_region.mode=="kl" else None,
        "dlogits_l2": l2_delta_of(action, logits_base)    if
cfg.trust_region.mode=="dlogits" else None,
        "candidates": len(C),
    }
    return action, audit
```

Fallbacks. If features missing or budgets exhausted → return **baseline sample** and log a `fallback=true` flag.

B.3 Trigger logic (BH-1, BH-2, micro-MPC)

```
def health_state(SSE, CRPz, SSIstarZ, thresholds, ema=0.75):
    Xi = ( thresholds.a * (1 - SSE)
         + thresholds.b * CRPz
         + thresholds.c * SSIstarZ )
    bh1 = (SSE <= thresholds.sse) and (CRPz >= thresholds.crp_z)   # short
window with debounce outside
    bh2 = Xi >= thresholds.xi                                      # sustained
in a medium window elsewhere
    if bh2: return "Emergency", bh1, bh2, Xi
    if bh1: return "Action",    bh1, bh2, Xi
    if Xi > 0.35: return "Caution", bh1, bh2, Xi
    return "Green", bh1, bh2, Xi

def should_micro_mpc(state, Es_spike, schema_boundary):
    return Es_spike or (state in {"Action","Emergency"}) or schema_boundary
```

B.4 Trust-region enforcement

Two simple, stable options: **KL** constraint or **Δlogits L2** constraint.

```
def project_distribution_kl(logits_base, logits_adj, kl_budget, tol=1e-4):
    """
    Temperature scaling via binary search to satisfy KL(q || p) <= kl_budget.
    """
    p = softmax(logits_base)
    q = softmax(logits_adj)
```

```
    if kl(q, p) <= kl_budget: return logits_adj

    # Scale down adjustment by temperature τ>=1
    lo, hi = 1.0, 32.0
    for _ in range(20):
        tau = 0.5 * (lo + hi)
        q_tau = softmax((logits_base + (logits_adj - logits_base)/tau))
        if kl(q_tau, p) <= kl_budget: hi = tau
        else: lo = tau
    tau = hi
    return logits_base + (logits_adj - logits_base)/tau

def clip_dlogits_l2(logits_base, logits_delta, l2_budget):
    norm = np.linalg.norm(logits_delta)
    if norm <= l2_budget or norm == 0: return logits_base + logits_delta
    return logits_base + logits_delta * (l2_budget / norm)

def argmax_under_trust_region(scored, logits_base, budgets, tr_cfg):
    # scored: list of (u, J, L, Gamma)
    u_star, _, _, _ = max(scored, key=lambda x: x[1])
    logits_adj = apply_bias(logits_base, u_star)   # bias towards chosen action
    if tr_cfg.mode == "kl":
        logits_proj = project_distribution_kl(logits_base, logits_adj,
budgets['kl'])
    else:
        logits_proj = clip_dlogits_l2(logits_base, logits_adj - logits_base,
budgets['dlogits'])
    return sample_from(logits_proj)
```

B.5 Optional: micro-MPC (short rollout)

```
def micro_mpc_rollouts(candidates, base_state, horizon=5, gamma=0.9,
budgets=None, cfg=None):
    """
    candidates : iterable of first actions u0
    base_state : light state struct (embeddings, parser state, last_modes,
fmt_debt, ...)
    returns    : list[(u0, Jbar, Lbar, Gammabar)]
    """
    results = []
```

```
    for u0 in candidates:
        st = base_state.copy()
        Lbar = Gbar = 0.0
        disc = 1.0
        ok_tr = True
        for h in range(horizon):
            logits_h = simulate_logits(st, u0 if h==0 else uh)      # cheap
rollout, not full beam
            uh = pick_top1(logits_h)                                # greedy
inside MPC
            Lh, Gh, st = estimate_step_scores_and_update(st, uh)    # cheap
updates
            Lbar += disc * Lh
            Gbar += disc * Gh
            disc *= gamma
            if budgets and exceeded_trust_region(st, budgets):
                ok_tr = False; break
        Jbar = Lbar - base_state.lambda_t * Gbar
        results.append((u0, -np.inf if not ok_tr else Jbar, Lbar, Gbar))
    return results
```

Use: Only when `should_micro_mpc()` is true. If all rollouts violate budgets or are NaN → **fallback to per-tick greedy**.

B.6 YAML profiles (ready to paste)

B6.1 `controller.v1.yaml`

```
controller:
  topk: 8
  horizon: 5
  gamma: 0.9
  micro_mpc_triggers: ["Es_spike", "BH1", "schema_boundary"]

  lambda:
    base: 0.4
    min: 0.0
    max: 2.0
    weights: { SSI: 0.8, CRP: 0.8, invSSE: 0.5 }   # λ = base * (1 + Σ w_i *
indicator_i)
    ema: 0.7
```

```
    slew_max: 0.25                                  # max absolute change per
minute

  trust_region:
    mode: "kl"                                      # or "dlogits"
    kl_budget_green: 0.010
    kl_budget_action: 0.006
    dlogits_green: 1.0
    dlogits_action: 0.7
    step_down_pct: 0.3                               # tighten this much in
Action/Emergency

  weights:
    alpha:                                          # L components
      dlp: 1.0
      progress: 0.6
      structRisk: 0.8
      latency: 0.3
      length: 0.2
    beta:                                           # Γ components (Γ-Lite
defaults)
      drift: 1.0
      flip: 0.7
      debt: 0.6

  guards:
    min_SSE: 0.50
    exploration_quota_pct: 0.15
    min_ticks_between_flips: 3
    fmt_debt_repair_threshold: 0.60
```

B6.2 `telemetry.v1.yaml`

```
telemetry:
  windows:
    short_s: 120
    medium_min: 10
    long_h: 4
  smoothing_ema: 0.75
  debounce_ticks: 3
```

```
  thresholds:
    sse: 0.45
    crp_z: 0.8
    xi: 0.65
    bands: {green: 0.35, caution: 0.55, emergency: 0.75}

  exports:
    cap_stream: "kafka://cap.v1"
    health_stream: "kafka://health.v1"
    retention_days: 14
    redact_pii: true
```

B6.3 `incidents.v1.yaml`

```
incidents:
  auto_actions:
    BH1: ["raise_lambda", "tighten_trust_region", "enable_micro_mpc",
"enforce_exploration", "freeze_flips"]
    BH2: ["route_throttle", "degrade_tools", "enable_HITL",
"suspend_hot_tenant_optional"]
  exit_criteria:
    sse: {min: 0.55, duration_min: 30}
    crp_z: {max: 0.3, duration_min: 30}
  rollback_ladder: ["R0_soft_nudge", "R1_structural_guards",
"R2_route_throttle", "R3_degrade", "R4_HITL", "R5_suspend", "R6_full_revert"]
```

B6.4 `policy.v1.yaml`

```
policy:
  caps:
    qps: 30
    tok_max: 15000
    tool_per_min: 40
  routing:
    min_spread: 0.55
    providers: ["P1", "P2"]
    max_provider_share: 0.40
  exploration:
    min_SSE: 0.50
    quota_pct: 0.15
  privacy:
```

```
    pii_redact: true
    retention_days: 14
    store_raw_text: false
```

B6.5 `clusters.v1.yaml` (for SSE)

```
clusters:
  method: "kmeans_ensemble"
  k_choices: [16, 24, 32]
  ensemble_size: 3
  reseed_days: 7
  drift_reseed_threshold: 0.12   # centroid movement (cosine) over a week
  knn_entropy_fallback_k: 25
```

B6.6 `tenants.v1.yaml` (overrides)

```
tenants:
  "acme":
    controller_overrides:
      lambda: { base: 0.5, max: 2.5 }
      trust_region: { kl_budget_green: 0.012, kl_budget_action: 0.008 }
    policy_overrides:
      caps: { qps: 50, tok_max: 25000 }
      routing: { max_provider_share: 0.35 }
```

B.7 Minimal unit tests (runnable skeleton)

```
def test_trust_region_enforced():
    logits = np.array([0.1, 0.0, -0.2, 0.3])
    adj    = logits + np.array([2.0, -1.0, 0.5, -1.5])  # big nudge
    proj   = project_distribution_kl(logits, adj, kl_budget=0.01)
    assert kl(softmax(proj), softmax(logits)) <= 0.011

def test_gamma_lite_monotonicity():
    g1 = gamma_lite(theta_t=0.2, theta_tp1=0.25, leak_t=0.1, leak_tp1=0.1,
last_modes=['A','B','A'], fmt_debt=0.2)
    g2 = gamma_lite(theta_t=0.2, theta_tp1=0.35, leak_t=0.1, leak_tp1=0.2,
last_modes=['A','B','A'], fmt_debt=0.2)
    assert g2 > g1   # more drift/leak ⇒ higher Γ

def test_schema_guard_injection():
    fmt_debt = 0.7
```

```
    repaired = (fmt_debt > 0.6)
    assert repaired
    # Next tick should emit structure token and drop debt
```

B.8 Integration checklist

Insert controller **between logits and sampling**; preserve a clean fallback path.

Emit **CAP** (controller audit packet) and **health** snapshots per §11.2 schemas.

Wire **BH-1/BH-2** triggers to auto-actions and rollback ladder.

Nightly **weight calibration** and SSE re-seeding per `clusters.v1.yaml`.

Privacy defaults: **no raw text** in telemetry; hashed embeddings/logits only.

Canary kill-switches tied to incident thresholds.

What this gives you. A production-ready skeleton: a cheap Γ you can ship today, event triggers that buy lead time, safe trust-region bounds, and YAML profiles that keep ops, safety, and governance aligned—without touching base model weights.

Appendix C — Telemetry Schemas, SQL Snippets, Dashboards

This appendix gives concrete, copy-pastable schemas and SQL for a minimal telemetry stack, plus dashboard layouts that on-call and governance can actually use. It assumes a streaming bus (e.g., Kafka) → warehouse (Postgres/BigQuery) flow. Privacy defaults to **metrics-only**; no raw text is stored.

C.1 Event Schemas (streaming)

C1.1 Controller Audit Packet (per tick) — `cap.v1` (JSON Schema)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "smfc-telemetry/cap.v1",
  "type": "object",
  "required":
["ts","tenant","session","model","hash_logits","theta","phi","iT","L","Gamma","la
  "properties": {
    "ts":            {"type": "string", "format": "date-time"},
    "tenant":        {"type": "string"},
    "session":       {"type": "string"},
    "model":         {"type": "string"},
    "hash_logits":   {"type": "string"},     // metrics only; no raw logits
```

```
"hash_embed":    {"type": "string"},
"theta":         {"type": "number"},
"phi":           {"type": "number"},
"iT":            {"type": "number"},
"Es":            {"type": "number"},
"L": {
  "type":"object",
  "properties":{
    "dlp":{"type":"number"},
    "progress":{"type":"number"},
    "structRisk":{"type":"number"},
    "lat":{"type":"number"},
    "len":{"type":"number"}
  }
},
"Gamma": {
  "type":"object",
  "properties":{
    "drift":{"type":"number"},
    "modeFlip":{"type":"number"},
    "fmtDebt":{"type":"number"},
    "iT":{"type":"number"}
  }
},
"lambda":        {"type": "number"},
"kl_used":       {"type": ["number","null"]},
"dlogits_l2":    {"type": ["number","null"]},
"action": {
  "type":"object",
  "properties":{
    "type":{"type":"string"},           // token|tool|plan
    "name":{"type":"string"},
    "args_hash":{"type":"string"}
  }
},
"candidates":    {"type":"integer"},
"caps_hit":      {"type":"boolean"},
"state":         {"type":"string"},     // Green|Caution|Action|Emergency
"bh1":           {"type":"integer"},    // 0/1
"bh2":           {"type":"integer"}     // 0/1
```

```
  }
}
```

C1.2 Health Snapshot (per minute) — `health.v1`

```
{
  "$id": "smfc-telemetry/health.v1",
  "type": "object",
  "required":
["bucket","tenant","SSE","CRPz","SSIstarZ","Xi","lambda","kl_budget","dlogits_buc
  "properties": {
    "bucket":         {"type":"string","format":"date-time"},   // minute
aligned
    "tenant":         {"type":"string"},
    "SSE":            {"type":"number"},
    "CRPz":           {"type":"number"},
    "SSIstarZ":       {"type":"number"},
    "Xi":             {"type":"number"},
    "lambda":         {"type":"number"},
    "kl_budget":      {"type":"number"},
    "dlogits_budget": {"type":"number"},
    "state":          {"type":"string"},
    "bh1":            {"type":"integer"},
    "bh2":            {"type":"integer"}
  }
}
```

C1.3 Incident Record — `incident.v1`

```
{
  "$id": "smfc-telemetry/incident.v1",
  "type": "object",
  "required": ["id","tenant","started_at","kind","severity","summary"],
  "properties": {
    "id":         {"type":"string"},
    "tenant":     {"type":"string"},
    "started_at": {"type":"string","format":"date-time"},
    "ended_at":   {"type":["string","null"],"format":"date-time"},
    "kind":       {"type":"string"},     // schema_break|latency_spike|
policy_violation|other
    "severity":   {"type":"string"},     // low|med|high|critical
```

```
    "summary":    {"type":"string"},
    "bh2_linked": {"type":"boolean"},
    "rollback_rung":{"type":"string"}    // R0..R6 if applied
  }
}
```

C.2 Warehouse Tables (Postgres DDL)

> If using BigQuery, swap types: `timestamptz → TIMESTAMP`, `jsonb → JSON`, and drop indexes in favor of partitioning.

```
-- C2.1 Raw CAP events
CREATE TABLE cap_events (
  ts           timestamptz NOT NULL,
  tenant       text        NOT NULL,
  session      text        NOT NULL,
  model        text        NOT NULL,
  hash_logits  text        NOT NULL,
  hash_embed   text,
  theta        double precision,
  phi          double precision,
  iT           double precision,
  Es           double precision,
  L            jsonb,              -- {dlp,progress,structRisk,lat,len}
  Gamma        jsonb,              -- {drift,modeFlip,fmtDebt,iT}
  lambda       double precision,
  kl_used      double precision,
  dlogits_l2   double precision,
  action       jsonb,              -- {type,name,args_hash}
  candidates   integer,
  caps_hit     boolean,
  state        text,
  bh1          smallint,
  bh2          smallint,
  -- optional reservoirs for SSI (per-event proxies)
  m_util       double precision,   -- material utilization 0..1 (optional)
  f_util       double precision,   -- financial
  i_util       double precision,   -- institutional
  a_util       double precision,   -- attention
  c_util       double precision    -- cognition
) PARTITION BY RANGE (ts);
```

```
CREATE INDEX ON cap_events (tenant, ts);
CREATE INDEX ON cap_events (session);

-- Daily partitions helper:
CREATE TABLE cap_events_2025_09_07 PARTITION OF cap_events
FOR VALUES FROM ('2025-09-07') TO ('2025-09-08');

-- C2.2 Health minutes (aggregated)
CREATE TABLE health_minutes (
  bucket        timestamptz NOT NULL,   -- minute bucket
  tenant        text        NOT NULL,
  SSE           double precision,
  CRPz          double precision,
  SSIstarZ      double precision,
  Xi            double precision,
  state         text,
  bh1           integer,
  bh2           integer,
  lambda        double precision,
  kl_budget     double precision,
  dlogits_budget double precision
);
CREATE UNIQUE INDEX ON health_minutes (tenant, bucket);

-- C2.3 Cluster assignments for SSE (rolling window)
CREATE TABLE sse_assignments (
  ts      timestamptz NOT NULL,
  tenant  text        NOT NULL,
  session text        NOT NULL,
  cluster integer     NOT NULL         -- 0..N-1
);
CREATE INDEX ON sse_assignments (tenant, ts);

-- C2.4 Incidents
CREATE TABLE incidents (
  id          text PRIMARY KEY,
  tenant      text NOT NULL,
  started_at  timestamptz NOT NULL,
  ended_at    timestamptz,
  kind        text NOT NULL,
```

```
  severity    text NOT NULL,
  summary     text,
  bh2_linked  boolean,
  rollback_rung text
);
```

C.3 SQL Snippets

C3.1 Minute rollup from CAP events → Health snapshot

```
WITH minute_caps AS (
  SELECT
    date_trunc('minute', ts) AS bucket,
    tenant,
    /* --- SSI proxy: take max across five per-event reservoir utilizations ---
*/
    GREATEST(
      PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY COALESCE(m_util,0)),
      PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY COALESCE(f_util,0)),
      PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY COALESCE(i_util,0)),
      PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY COALESCE(a_util,0)),
      PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY COALESCE(c_util,0))
    ) AS SSIstar,                            -- 0..1

    /* --- CRP proxy: backlog growth & rework --- */
    AVG( (Gamma->>'fmtDebt')::float )                                  AS
fmt_debt_avg,
    AVG( (Gamma->>'modeFlip')::float )                                 AS flip_avg,
    STDDEV_POP( (L->>'lat')::float )                                   AS lat_sd,
    AVG( CASE WHEN (L->>'structRisk')::float > 0.5 THEN 1 ELSE 0 END ) AS
rework_rate,

    SUM(bh1)::int AS bh1_cnt,
    SUM(bh2)::int AS bh2_cnt
  FROM cap_events
  WHERE ts >= now() - interval '2 hours'
  GROUP BY 1,2
),
crp_norm AS (
  SELECT
    m.*,
```

```
    /* z-score each tenant using 6h trailing window */
    (m.rework_rate - AVG(m.rework_rate) OVER w)
      / NULLIF(STDDEV_POP(m.rework_rate) OVER w,0)   AS rework_z,
    (m.lat_sd - AVG(m.lat_sd) OVER w)
      / NULLIF(STDDEV_POP(m.lat_sd) OVER w,0)        AS latvar_z,
    (m.flip_avg - AVG(m.flip_avg) OVER w)
      / NULLIF(STDDEV_POP(m.flip_avg) OVER w,0)      AS flip_z
  FROM minute_caps m
  WINDOW w AS (PARTITION BY tenant ORDER BY bucket
               RANGE BETWEEN INTERVAL '6 hour' PRECEDING AND CURRENT ROW)
),
sse_calc AS (
  /* compute SSE from cluster shares in the same minute bucket */
  SELECT
    date_trunc('minute', ts) AS bucket,
    tenant,
    - SUM( (cnt::float/NULLIF(sum_cnt,0)) * LN(cnt::float/NULLIF(sum_cnt,0)) ) /
LN(NULLIF(n_clusters,1)) AS SSE
  FROM (
    SELECT
      sa.tenant,
      sa.ts,
      sa.cluster,
      COUNT(*) AS cnt,
      SUM(COUNT(*)) OVER (PARTITION BY sa.tenant, date_trunc('minute',sa.ts)) AS
sum_cnt,
      COUNT(DISTINCT sa.cluster) OVER (PARTITION BY sa.tenant,
date_trunc('minute',sa.ts)) AS n_clusters
    FROM sse_assignments sa
    WHERE sa.ts >= now() - interval '2 hours'
    GROUP BY sa.tenant, sa.ts, sa.cluster
  ) x
  GROUP BY 1,2
)
INSERT INTO health_minutes (bucket, tenant, SSE, CRPz, SSIstarZ, Xi, state, bh1,
bh2, lambda, kl_budget, dlogits_budget)
SELECT
  c.bucket,
  c.tenant,
  COALESCE(s.SSE, 0.7) AS SSE,
```

```
  /* CRPz: combine z-scores */
  COALESCE(0.5*c.rework_z + 0.3*c.latvar_z + 0.2*c.flip_z, 0.0) AS CRPz,
  /* SSIstarZ: z-score SSIstar in 6h window */
  (c.SSIstar - AVG(c.SSIstar) OVER w) / NULLIF(STDDEV_POP(c.SSIstar) OVER w,0)
AS SSIstarZ,
  /* composite risk Xi = a*(1-SSE) + b*CRPz + c*SSIstarZ */
  (0.6*(1-COALESCE(s.SSE,0.7)) + 0.8*COALESCE(0.5*c.rework_z + 0.3*c.latvar_z +
0.2*c.flip_z,0.0) + 0.8*((c.SSIstar - AVG(c.SSIstar) OVER w)/
NULLIF(STDDEV_POP(c.SSIstar) OVER w,0))) AS Xi,
  /* state bands */
  CASE
    WHEN (0.6*(1-COALESCE(s.SSE,0.7)) + 0.8*COALESCE(0.5*c.rework_z +
0.3*c.latvar_z + 0.2*c.flip_z,0.0) + 0.8*((c.SSIstar - AVG(c.SSIstar) OVER w)/
NULLIF(STDDEV_POP(c.SSIstar) OVER w,0))) >= 0.75 THEN 'Emergency'
    WHEN (0.6*(1-COALESCE(s.SSE,0.7)) + 0.8*COALESCE(0.5*c.rework_z +
0.3*c.latvar_z + 0.2*c.flip_z,0.0) + 0.8*((c.SSIstar - AVG(c.SSIstar) OVER w)/
NULLIF(STDDEV_POP(c.SSIstar) OVER w,0))) >= 0.55 THEN 'Action'
    WHEN (0.6*(1-COALESCE(s.SSE,0.7)) + 0.8*COALESCE(0.5*c.rework_z +
0.3*c.latvar_z + 0.2*c.flip_z,0.0) + 0.8*((c.SSIstar - AVG(c.SSIstar) OVER w)/
NULLIF(STDDEV_POP(c.SSIstar) OVER w,0))) >= 0.35 THEN 'Caution'
    ELSE 'Green'
  END AS state,
  SUM(c.bh1_cnt) OVER (PARTITION BY c.tenant, c.bucket) AS bh1,
  SUM(c.bh2_cnt) OVER (PARTITION BY c.tenant, c.bucket) AS bh2,
  /* controller budgets (example rules): derive λ and budgets here or in app */
  NULL::double precision AS lambda,
  NULL::double precision AS kl_budget,
  NULL::double precision AS dlogits_budget
FROM crp_norm c
LEFT JOIN sse_calc s
  ON s.bucket = c.bucket AND s.tenant = c.tenant
WINDOW w AS (PARTITION BY c.tenant ORDER BY c.bucket
             RANGE BETWEEN INTERVAL '6 hour' PRECEDING AND CURRENT ROW)
ON CONFLICT (tenant, bucket) DO UPDATE
SET SSE = EXCLUDED.SSE, CRPz = EXCLUDED.CRPz, SSIstarZ = EXCLUDED.SSIstarZ,
    Xi = EXCLUDED.Xi, state = EXCLUDED.state, bh1 = EXCLUDED.bh1, bh2 =
EXCLUDED.bh2;
```

C3.2 Detect BH-1 / BH-2 with window functions (for alerts)

```
-- BH-1: low SSE AND high CRPz in short window (debounced)
```

```
SELECT tenant, bucket,
       (MIN(SSE)  FILTER (WHERE bucket >= now()-interval '3 minutes') <= 0.45
    AND MAX(CRPz) FILTER (WHERE bucket >= now()-interval '3 minutes') >= 0.8) AS
BH1_now
FROM health_minutes
WHERE bucket >= now() - interval '15 minutes'
GROUP BY tenant, bucket;

-- BH-2: Xi ≥ τ_xi sustained N of last M minutes
WITH last_m AS (
  SELECT tenant, bucket, Xi,
         COUNT(*) OVER (PARTITION BY tenant ORDER BY bucket
                        RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND
CURRENT ROW) AS m_count,
         SUM( CASE WHEN Xi >= 0.65 THEN 1 ELSE 0 END ) OVER
             (PARTITION BY tenant ORDER BY bucket
              RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW) AS
ge_count
  FROM health_minutes
  WHERE bucket >= now() - interval '30 minutes'
)
SELECT tenant, bucket, (ge_count >= 5 AND m_count >= 10) AS BH2_sustained
FROM last_m;
```

C3.3 Lead-time between BH alerts and incidents

```
-- For each incident, find the most recent BH-1 and BH-2 before it and compute
lead times.
WITH bh1 AS (
  SELECT tenant, bucket AS ts FROM health_minutes
  WHERE bh1 > 0
), bh2 AS (
  SELECT tenant, bucket AS ts FROM health_minutes
  WHERE bh2 > 0
)
SELECT i.id, i.tenant, i.started_at,
       EXTRACT(EPOCH FROM (i.started_at - (SELECT max(ts) FROM bh1 b WHERE
b.tenant=i.tenant AND b.ts<=i.started_at))) / 60.0 AS lead_min_bh1,
       EXTRACT(EPOCH FROM (i.started_at - (SELECT max(ts) FROM bh2 b WHERE
b.tenant=i.tenant AND b.ts<=i.started_at))) / 60.0 AS lead_min_bh2
FROM incidents i
```

```
WHERE i.started_at >= now() - interval '30 days';
```

C3.4 Mode-flip rate, format-debt trend, and λ coupling sanity

```
-- Hourly rollups for ops dashboards
SELECT date_trunc('hour', ts) AS hour,
       tenant,
       AVG( (Gamma->>'modeFlip')::float ) AS flip_rate,
       AVG( (Gamma->>'fmtDebt')::float )  AS fmt_debt,
       AVG(lambda)                        AS lambda_avg,
       AVG(CASE WHEN state='Action' THEN 1 ELSE 0 END) AS action_duty
FROM cap_events
WHERE ts >= now() - interval '48 hours'
GROUP BY 1,2
ORDER BY 1,2;
```

C.4 Dashboards (layout & queries)

C4.1 On-Call "Live Health" (Grafana/Metabase)

Top strip (per tenant toggle)

SSE, CRPz, SSIstarZ (3 time-series, last 2h, band shading).

Xi with state colors (Green/Caution/Action/Emergency).
Query: `SELECT bucket, SSE, CRPz, SSIstarZ, Xi FROM health_minutes WHERE tenant=:tenant AND bucket>=now()-interval '2 hours' ORDER BY bucket;`

Alerts & Lead Time

BH-1/BH-2 markers on Xi chart (event overlay).

Table: last 10 incidents with lead_min_bh1/bh2 (from C3.3).

Controller Internals

λ vs. KL/Δlogits budget usage (%).

Mode-flip rate, FmtDebt trend (C3.4).

Percent of ticks where **micro-MPC** triggered (if logged as a flag in CAP `action`).

Capacity & Caps

p95 latency, tokens/min, tool_ms/min, caps_hit rate.

Utilization fan chart if reservoir proxies are available (m/f/i/a/c).

Alert rules (examples):

Xi ≥ 0.65 sustained 10m → page **SEV-2**.

SSE < 0.45 and CRPz > 0.8 for 3 consecutive mins → auto-apply BH-1 runbook actions.

caps_hit > 5% for 10m → send to capacity channel.

C4.2 Governance & Equity

Panels:

Bias Gap Index (BGI) per cohort (if available): pass rate differences with CI bands.

Payout ledger summary (from credit system): total, novelty-weighted, uplift-weighted.

Incident disclosure feed: last 30d incidents with root cause and rollback rung.

C4.3 Post-Incident Review (blameless)

Panels:

Timeline: Xi, SSE, CRPz, SSIstarZ, λ, BH marks, caps toggles.

Decision table: minute-by-minute changes to λ, budgets, routing/caps.

Counterfactuals: shadow J vs. realized J for the window (if shadow stored).

Recovery metrics: time to exit Emergency → Action → Caution → Green; SSE floor recovery.

C.5 Privacy, Retention, and Partitioning

No raw text in telemetry; embeddings/logits stored as **hashes** only.

Tenant partitioning: per-tenant schemas or row-level security; exports require dual approval.

Retention: CAP 14 days, Health 60 days, Incidents 2 years (configurable).

Partitions: daily partitions on `cap_events.ts`; weekly/monthly on rollups.

PII redaction: if any IDs could be personal, store salted hashes; keep salt per tenant.

C.6 dbt Model Hints (optional)

`stg_cap_events` → cast JSON fields to columns (`L_progress`, `Gamma_fmtDebt`, …).

`f_health_minutes` → implements C3.1; scheduled every minute.

`f_alerts_bh` → implements BH-1/BH-2 detections.

`f_leadtime` → implements C3.3; refreshed hourly.

Tests: `not_null`, `unique` on `(tenant,bucket)`; `accepted_values` on `state`.

C.7 Quick Validation Queries

Are bands working?

```
SELECT state, COUNT(*) FROM health_minutes
WHERE bucket >= now() - interval '24 hours'
GROUP BY 1 ORDER BY 2 DESC;
```

Do BH-1 precede BH-2?

```
SELECT tenant,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY lead_min_bh1) AS p50_lead_bh1,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY lead_min_bh2) AS p50_lead_bh2
FROM ( /* C3.3 subquery here */ ) x
GROUP BY tenant;
```

Is λ responding to stress?

```
SELECT date_trunc('minute', ts) AS bucket, tenant,
       AVG(lambda) AS lambda_avg,
       AVG( CASE WHEN state IN ('Action','Emergency') THEN 1 ELSE 0 END ) AS
stress_duty
FROM cap_events
WHERE ts >= now() - interval '6 hours'
GROUP BY 1,2
ORDER BY 1;
```

Outcome. These schemas, tables, and queries give you a working telemetry backbone: per-tick audit, per-minute health, crisp BH-1/BH-2 alerts, and dashboards that surface **lead time**, **controller behavior**, and **capacity stress** without exposing raw content.

Appendix D — Sample Data-Sharing / Compensation Clauses (Engineering Summary)

> This appendix offers **drop-in, implementation-ready** clauses and artifacts your legal team can adapt. Everything maps to the telemetry and ledger objects defined earlier (no raw text required). Square-bracket fields **[like this]** are fill-ins.

D.1 Scope & Definitions

Purpose. Enable privacy-preserving data contribution for improvement, evaluation, and safety, with transparent compensation and revocation.

Key terms.

Contribution: Any dataset, interaction, annotation, or evaluation artifact provided by a **Contributor** (individual or organization).

Contribution ID (`contrib_id`): A stable, hashable provenance bundle for a Contribution.

Consent: `active | scoped | revoked`; scope includes purpose, time, regions, redistribution.

Derived Artifact: Model weights, metrics, or anonymized aggregates created from Contributions.

Ledger: The append-only record of usage, uplift, novelty, safety scalar, and payouts (see `credit.v1` in §11.2).

Implementation note. Use content fingerprints + tenant salt to generate `contrib_id`. Never store raw content in the Ledger—only hashes and metrics.

D.2 Acceptable Use & Prohibited Content

Clause. "Provider SHALL NOT ingest Contributions containing: (i) government-issued identifiers, (ii) sensitive health/biometric data, (iii) minors' personal data, (iv) secrets/credentials, (v) IP where Contributor lacks rights. Provider SHALL run PII redaction and secret-scan before acceptance."

Engineering hook.

```
ingestion_policy:
  pii_redaction: true
  secret_scanner: true
  prohibited_tags: ["gov_id","minors","biometric","credentials","no_rights"]
  rejection_mode: "quarantine"   # quarantine -> notify -> purge in T=7d if
unremedied
```

D.3 Consent & Purpose Limitation

Clause. "Provider MAY use Contributions for [training|evaluation|safety|replay] within [regions], for [duration]. Redistribution to third parties is prohibited except federated aggregates meeting k-anonymity ≥ k* and DP-epsilon ≤ ε*."

API contract (Consent object).

```
{
  "contrib_id": "cid:sha256:…",
  "owner": "org:AcmeLabs",
  "consent": "scoped",
  "scope": {"purpose":["evaluation","safety"], "regions":["EEA","US"],
"ttl_days": 365},
  "flags": {"allow_federated": true, "allow_rerelease": false, "dp_epsilon_max":
2.0}
```

```
}
```

D.4 Provenance & Traceability

Clause. "Each Contribution MUST retain `contrib_id` through all pipelines; Derived Artifacts MUST embed a **contribution vector** (counted usage weights) and be auditable to minute-level buckets."

Engineering hook.

```
{
  "model_release": "controller-1.2.0",
  "contrib_vector": [{"contrib_id":"cid:…","w":0.0012},
{"contrib_id":"cid:…","w":0.0007}],
  "provenance_hash": "h:merkle:…"
}
```

D.5 Compensation Formula & Examples

Clause. "Provider SHALL compensate Contributors monthly using a transparent metric-based formula."

Formula (same as §9, explicit here).

$$\text{Payout}_i = r_{\text{base}} \cdot \left(u_i + w_q \, \Delta Q_i + w_n \, \text{Novelty}_i\right) \cdot s_i$$

u_i: normalized usage (queries/tokens/coverage).

ΔQ_i: uplift attributable to i (A/B or replay).

Novelty_i: de-dup/uniqueness score $\in [0, 1]$.

s_i: safety scalar $\in [0, 1]$ (down-weights risky content).

r_{base}, w_q, w_n: published coefficients.

Published coefficients (example).

```
payout_v1:
  r_base: 0.005     # $/unit
  w_q:    2.0
  w_n:    0.5
  floors: {payout_min_usd: 5.00}
  caps:   {per_month_usd: 25000}
```

Ledger entry (example).

```
{
```

```
  "contrib_id": "cid:…",
  "period": "2025-08",
  "usage": {"queries": 12190, "tokens": 4_320_000, "coverage": 0.27, "u":
137.5},
  "uplift": {"delta_quality": 0.06, "ci": [0.03,0.09]},
  "novelty": 0.72,
  "safety_scalar": 0.93,
  "payout_usd": 98.44,
  "explain": "u=137.5, ΔQ=0.06 → w_q*ΔQ=0.12, novelty=0.72→w_n*nov=0.36; s=0.93"
}
```

Attribution method. If multiple Contributions overlap a run, allocate uplift proportionally to Shapley-like weights computed on replay subsets; publish method summary and seed.

D.6 Payment Schedule & Taxes

Clause. "Payouts accrue monthly and are disbursed within **T=30 days** of period end. Provider furnishes tax forms as required. Negative adjustments SHALL NOT exceed the month's accruals."

D.7 Revocation, Deletion & Cooling-Off

Clause. "Contributor MAY revoke consent any time. Provider SHALL (i) cease new use within **T_0=24h**, (ii) purge caches and training shards within **T_1=7d**, and (iii) remove from replay/eval within **T_2=30d**. Derived Artifacts remain unless uniquely deanonymizing; in such cases, Provider SHALL retrain from the last clean snapshot."

Engineering hook.

```
revocation:
  stop_new_use_hours: 24
  purge_caches_days: 7
  purge_replay_days: 30
  backfill_snapshot_policy: "nightly_clean"
```

D.8 Incident Handling & Disclosure

Clause. "On material misuse or breach, Provider SHALL notify within **72h**, publish an incident summary within **7d**, and credit affected Contributors with a buffer-fund stipend defined in policy."

Linkage. Use `incident.v1` (Appendix C) and include `bh2_linked`, `rollback_rung`.

D.9 Federated Exchange & Portability

Clause. “Federated partners MAY exchange **metrics, gradients, or DP-aggregates** only. Raw Contributions remain local. Credits are portable: Contributors MAY transfer balances to another Provider at fair value minus exchange fee ≤ **x%**.”

Engineering hook.

```
federation:
  exchange_objects: ["dp_metric","secure_agg_gradient"]
  dp: {epsilon_max: 2.0, delta: 1e-6}
  secure_agg: {k_anonymity_min: 50}
  portability_fee_pct: 1.0
```

D.10 Evaluation Access (Read-Only)

Clause. “Provider MAY retain redacted, hashed telemetry for evaluation and safety for **60 days**; no raw text. Exports require dual-control approval and purpose logging.”

RBAC policy (example).

```
rbac:
  roles:
    auditor_ro: ["read:health_minutes","read:incidents","export:ledger"]
    oncall:     ["read:cap_events","read:health_minutes"]
  dual_control_exports: true
  retention_days:
    cap_events: 14
    health_minutes: 60
    incidents: 730
```

D.11 Bias & Equity Commitments

Clause. “Provider SHALL track **Bias Gap Index** (BGI) across cohorts and publish quarterly deltas. Where gap > τ, Provider SHALL fund targeted evaluation and adjust controller parameters to reduce associated dissipation.”

Engineering hook. Store BGI metrics alongside health; tie remedial actions to change logs.

D.12 IP, Licenses, and Warranty

Clause. “Contributor warrants rights to supply Contributions under [license]. Provider receives a non-exclusive, revocable license per consent scope. No transfer of Contributor IP except Derived Artifacts meeting DP/k-anonymity constraints.”

D.13 Change Management

Clause. “Material changes to payout coefficients, consent scope, or retention require 30-day notice and explicit re-consent for affected Contributors.”

Engineering hook. Version all policies; include `policy_version` in ledger rows; block ingestion when re-consent pending.

D.14 Term & Termination

Clause. “This agreement continues until revoked; termination triggers §D.7 revocation timelines and final payout settlement.”

D.15 Minimal Data-Sharing Addendum (1-page)

Cut-paste template (plain language).

```
1) What we collect: your contributed prompts, labels, and structured feedback.
2) Why: to improve evaluation, safety, and quality; not for unrelated
advertising.
3) How we protect it: redaction, hashing, no raw text in telemetry, strict
retention.
4) Your control: view ledger; revoke anytime; we stop new use in 24h and purge
in 7–30d.
5) Getting paid: monthly, formula published; you can export or port credits
elsewhere.
6) When things go wrong: we disclose incidents; you get buffer-fund stipend if
affected.
7) Who sees what: only metrics leave your region; raw data stays local unless
you opt-in.
```

D.16 End-to-End Flow (Swimlane)

```
Contributor → [Upload API] → Ingestion (PII/secret scan) → Consent check
→ Fingerprint → contrib_id → Storage (encrypted, scoped)
→ Replay/Eval jobs (metrics only) → Ledger update (usage, uplift, novelty,
safety)
→ Monthly Payout Engine → Statement to Contributor
→ (Optional) Federated DP aggregate → Partner (no raw)
→ Revocation? → Stop new use (24h) → Purge caches (7d) → Purge replay (30d)
```

D.17 API Stubs (for quick wiring)

Upload

```
POST /contrib/v1/upload
Headers: X-Owner: org:AcmeLabs
Body: { "contrib_hash":"h:…", "metadata":{"domain":"code","lang":"en"},
"consent":"scoped" }
→ 201 { "contrib_id":"cid:…", "status":"accepted|quarantine" }
```

Ledger Export

```
GET /ledger/v1/statement?owner=org:AcmeLabs&period=2025-08
→ 200 [ credit.v1 rows … ]
```

Revoke

```
POST /contrib/v1/revoke
Body: { "contrib_id":"cid:…", "reason":"owner_request" }
→ 202 { "stop_new_use_eta":"24h", "purge_by":"2025-10-01" }
```

D.18 Example Annex: Numbers That Fit

Revocation timers: T_0=24h (stop new), T_1=7d (cache purge), T_2=30d (replay purge).

DP budget: $\varepsilon \le 2.0$ per period, δ=1e-6.

k-anonymity: $k \ge 50$ on any external aggregate.

Retention: CAP 14d, Health 60d, Incidents 24mo.

Payout bounds: floor $5 / period; cap $25k / month / contributor.

What you can ship today

Adopt these clauses, wire the **Consent API**, generate `contrib_id`, and start the **Ledger** with the payout formula above.

Keep everything **metrics-only** in telemetry; publish coefficients and revocation SLAs.

Align payouts with uplift + novelty + safety to reinforce healthy, diverse data ecosystems.

Appendix E — Glossary of Terms

Task value (V)
The measurable utility created by a model action for its consumer within a bounded horizon. In practice: correctness, usefulness, progress toward a goal, format validity, and latency/length efficiency. Often appears inside the value term L of the per-step objective. *See also:* **L (value)**, **surplus**.

Surplus (S)
Net benefit after paying direct and indirect costs: $S = V - C_{\text{direct}} - C_{\text{indirect}}$. Positive surplus builds **residual** capacity; negative surplus erodes it and raises collapse risk. *See also:* **residual**, **collapse**.

Residual (R)
Carry-over capacity in a reservoir after an interaction:
$R_{t+1} = R_t + S_t - \text{obligations}_t - \text{buffer_topups}_t$. Tracked per flow $k \in \{M, F, I, A, C\}$: Material, Financial, Institutional, Attention, Cognition. *See also:* **reservoirs**, **SSI**.

Reservoirs (M, F, I, A, C)
Five coupled stocks that gate real deployments: compute/material (M), financial (F), institutional throughput (I), human attention (A), and cognitive/agent planning bandwidth (C). Each has capacity K_k, losses L_k, and leak μ_k. *See also:* **SSI**.

Collapse
A thresholded regime shift where residuals and buffers fall below resilience levels, causing self-reinforcing degradation (quality dips, latency spikes, backlog spirals). Characterized by hysteresis: recovery requires more than simply lowering load. *See also:* **CRP**, **BH-2**, **hysteresis**.

Attractor
A basin in semantic state space toward which outputs tend to settle (topic/style/plan that self-reinforces). Healthy attractors organize work; over-deep ones cause concentration and brittleness —"semantic black holes." *See also:* **semantic black hole**, **SSE**.

Semantic black hole
A narrow, self-reinforcing attractor with **low semantic spread entropy (SSE)**, high fragility (**CRP**), and rising saturation (**SSI**). Inside, behavior looks near-linear but is brittle. *See also:* **near-linearity**, **BH-1**.

Near-linearity
Locally linear response of outputs to small nudges within a deep attractor. Useful for gentle control, dangerous if mistaken for global linearity. *See also:* **semantic black hole**, **semantic mass**.

Orientation (θ)
Angle between the current embedding e_t and goal vector g: $\cos\theta = \langle e_t, g \rangle / (\| e_t \| \| g \|)$. Small θ means on-track; increases indicate drift. *See also:* **projection operator**, **progress**.

Projection operator ($\hat{\mathbf{0}}$)
Linear map projecting the current state onto the constraint subspace S (format/policy/persona). Produces in-subspace ratio ϕ and orthogonal leakage $\ell = \sqrt{1 - \phi^2}$. *See also:* **format debt**, **iT**.

Semantic tension (iT)
Stored misalignment/concentration pressure: $iT = \alpha(1 - \cos\theta) + \beta\,\ell + \gamma(1 - \text{SSE})$. High iT forecasts downstream rework; appears in the dissipation term Γ. *See also:* **Γ (dissipation)**.

Semantic mass (m_s)
Inertia of the content trajectory: resistance to steering. Estimated from impulse-to-acceleration: $m_s \approx I/(\Delta^2\theta/\Delta\tau^2)$. High $m_s \Rightarrow$ use structured steering (schemas), not tiny logit nudges. *See also:* **E_s**, **F_s**.

Semantic force (F_s)
Restoring "pull" toward the goal from a simple potential $U(\theta) = \frac{1}{2}\kappa(1-\cos\theta)^2$: $F_s = -\partial U/\partial\theta$. *See also:* **U(\theta)**, **progress**.

Semantic energy (E_s)
Run "hotness": $E_s = T_s + U(\theta)$ with $T_s = \frac{1}{2}m_s\omega^2$, $\omega = \Delta\theta/\Delta\tau$. Spikes are good triggers for micro-lookahead and tighter trust regions. *See also:* **micro-MPC**.

L (value)
Per-step value component in the objective: likelihood gain, progress toward goals/constraints, minus structure/latency/length penalties. Auditable from logs. *See also:* **J**, **Γ (dissipation)**.

Γ (dissipation)
Per-step predicted downstream burden: topic/intent drift, mode/tool chattering, format integrity debt, and *iT* carryover. High Γ correlates with rework and collapse risk. *See also:* **mode flip**, **format debt**.

J (objective)
The decoding controller's score for a candidate action: $\boxed{J = L - \lambda\Gamma}$. Maximized each tick under a stability constraint. *See also:* **λ**, **trust region**.

λ (lambda)
Trade-off weight between value and dissipation. Adapted online from health telemetry: $\lambda = f(\mathrm{SSI}, \mathrm{CRP}, \mathrm{SSE})$. Tightens under stress or concentration. *See also:* **SSI**, **CRP**, **SSE**.

SSI (Saturation–Stress Index)
0–1 capacity pressure signal combining utilization, backlog pressure, and cap hits (often max/p95 across shards). High SSI predicts throttling and drift. *See also:* **caps**, **collapse**.

CRP (Collapse Readiness Proxy)
Fragility indicator combining backlog growth, rework rate, turnaround variance, and near-misses (normalized). Rising CRP means small shocks will amplify. *See also:* **incidents**, **BH-2**.

SSE (Semantic Spread Entropy)
Normalized diversity of topics/plans over a rolling window: $\mathrm{SSE} = -\sum_i p_i \log p_i / \log N$. Low SSE flags concentration and black-hole formation. *See also:* **attractor**.

Composite risk (Ξ)
Single risk score: $\Xi = a(1-\mathrm{SSE}) + b\,\widetilde{\mathrm{CRP}} + c\,\widetilde{\mathrm{SSI}}^{\star}$. Drives BH-2 and policy escalations. *See also:* **BH-2**, **governance**.

BH-1 / BH-2 (black-hole alerts)
BH-1: early alert when SSE is low **and** CRP high (short window). BH-2: severe alert when Ξ breaches a sustained threshold (medium window). Wire to auto-actions and rollback. *See also:* **rollback ladder**.

Trust region
A stability bound constraining deviation from baseline logits: $\mathrm{KL}(q \,\|\, p) \le \varepsilon$ **or** $\|\Delta\text{logits}\|_2 \le \delta$. Prevents wild swings while the controller optimizes J. *See also:* **micro-MPC**.

Micro-MPC (micro model predictive control)
Short-horizon (3–10 tick) lookahead used when alarms fire or at schema edges. Chooses the branch with best discounted J subject to trust-region budgets. *See also:* **E_s**, **BH-1**.

Mode flip (tool/mode chattering)
Rapid A→B→A switching that burns latency and attention. Penalized inside Γ; guarded by dwell times and flip shields. *See also:* **Γ (dissipation)**.

Format debt (FmtDebt)
Accumulated probability that current output will require structural repair (JSON/sections/citations). Grows with borderline tokens; shrinks when repair tokens are emitted. *See also:* **projection operator**, **L (structRisk)**.

Exploration quota
Minimum fraction of traffic or decoding steps forced to diversify topics/plans/providers when SSE falls below a floor. Balances exploitation with spread. *See also:* **SSE**, **routing**.

Routing costs ("semantic currency")
Edge costs $\rho_{i \to j}$ charged for handoffs between agents/tools to discourage unnecessary coordination under stress. Added to Γ. *See also:* **mode flip**.

Caps
Operational ceilings per reservoir (QPS, tokens, tool calls, reviewer-hours, retries). Tightened automatically under high SSI/CRP; recorded in policy packs. *See also:* **SSI**, **rollback ladder**.

Rollback ladder (R0–R6)
Graded mitigation steps: soft nudge → structural guards → route/throttle → graceful degrade → human-in-the-loop → suspend tenant/feature → full revert. Each rung has exit criteria. *See also:* **BH-2**.

Hysteresis
Asymmetric transition: the load at which a system collapses differs from the load required to recover (due to depleted buffers/fatigue). *See also:* **collapse**, **buffers**.

Buffers / Buffer fund
Reserved reviewer time, rest windows, and contingency budget activated during high-risk phases (e.g., on BH-1/BH-2) to rebuild residuals and shorten recovery. *See also:* **CRP**, **governance**.

Controller Card
Release note documenting L/Γ terms, λ schedule, trust-region budgets, exploration quotas, routing costs, validation metrics, and known limitations. *See also:* **audit**, **governance**.

Audit packet (CAP)
Per-tick, privacy-preserving record of controller internals: L, Γ, λ, KL/Δlogits used, triggers, chosen action, caps hits, state. Enables incident reconstruction and external accountability. *See also:* **telemetry**.

Usage tip: When in doubt, tie a term to (i) **how it's computed**, (ii) **what it predicts**, and (iii) **what lever it moves**. This keeps the whole loop—metrics → control → governance—tight and auditable.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.